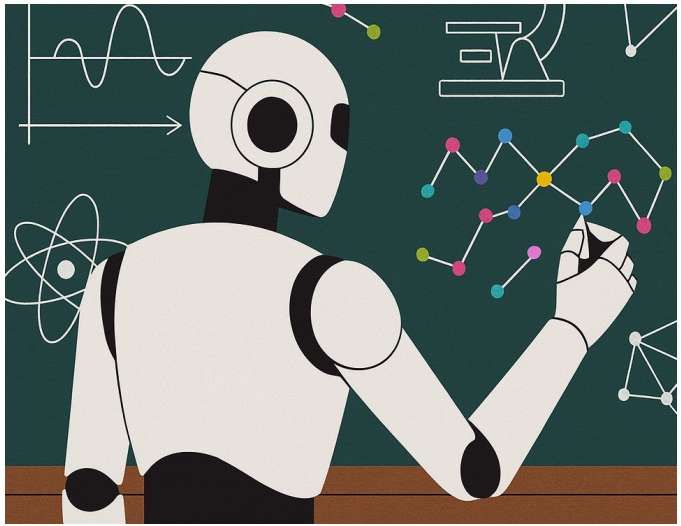
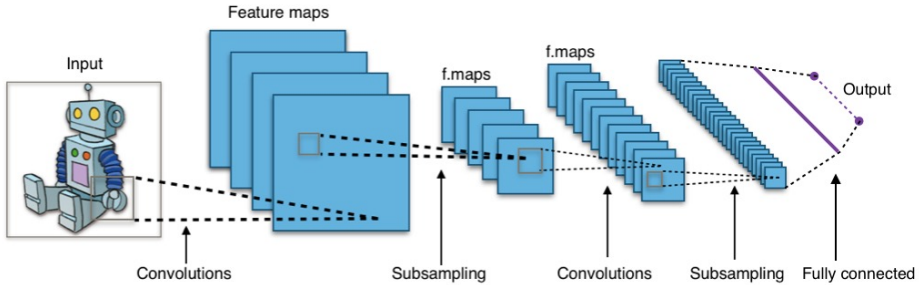
AI for Quantum Physics



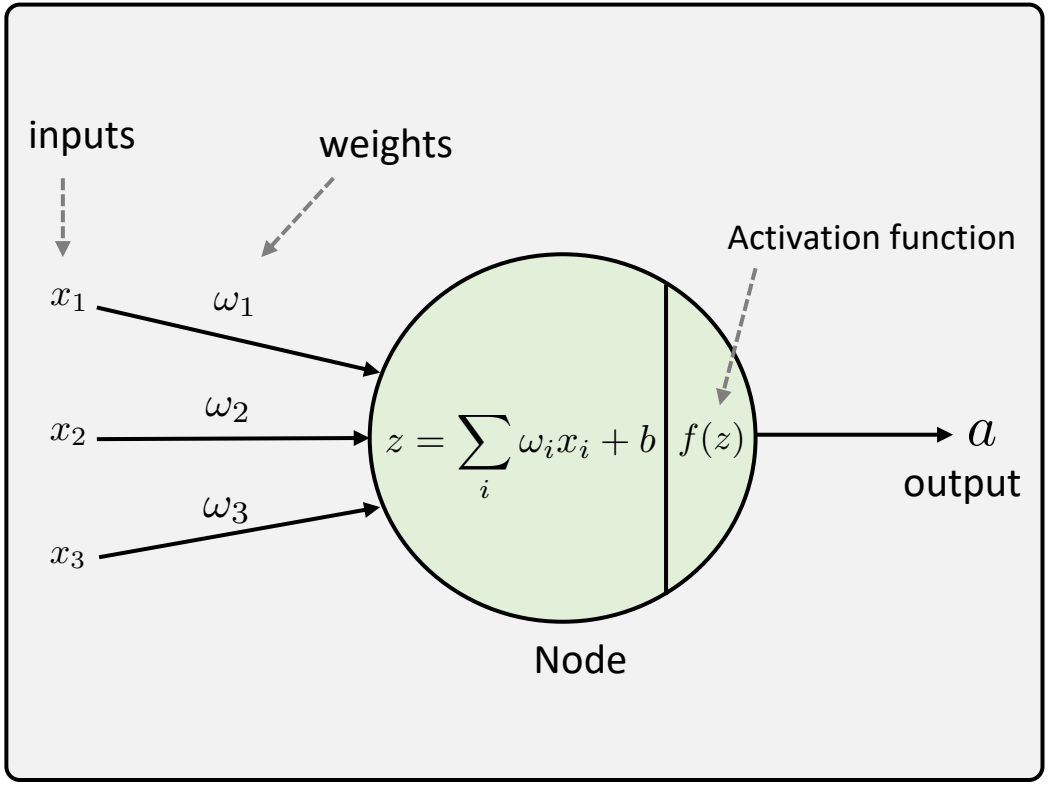
Xuemei Gu
xuemei.gu@uni-jena.de

Today's topic:

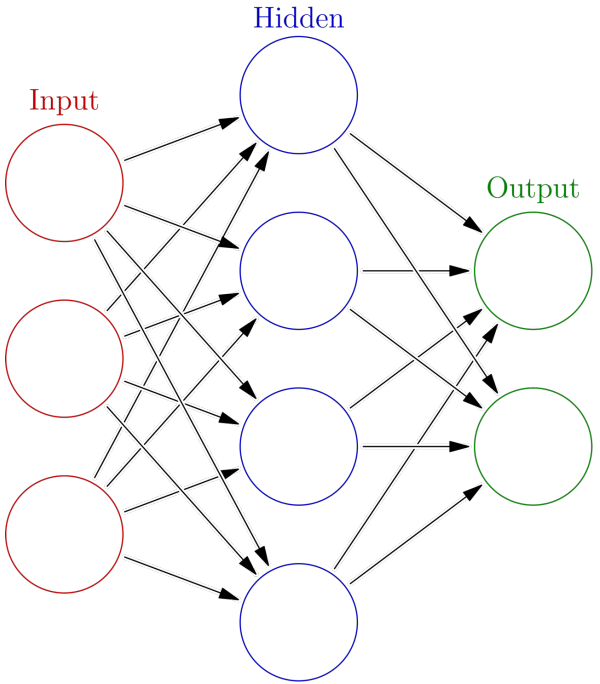
- Convolutional Neural Networks
- Examples
- Discussion



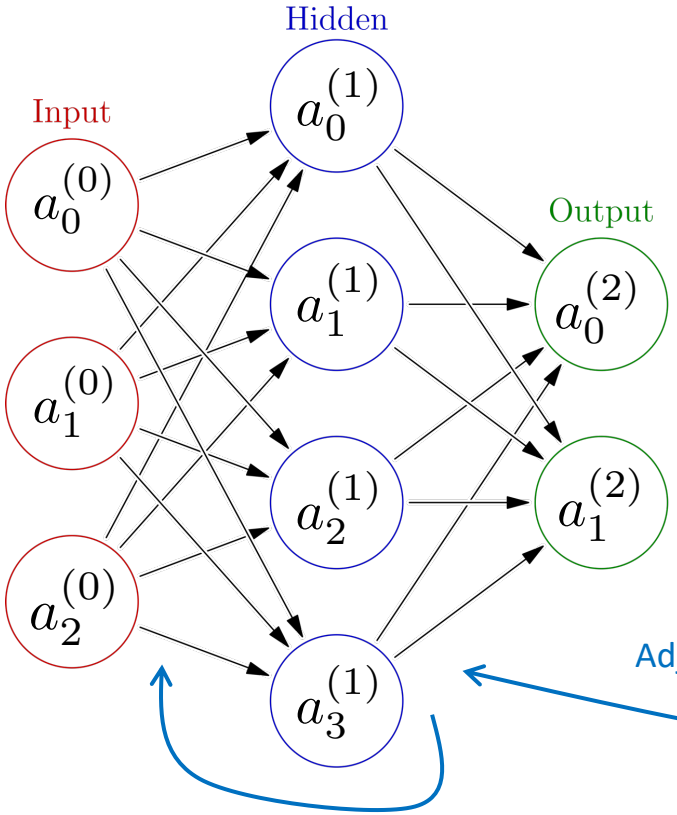
If you have any questions, feel free to reach out!



An artificial neural network (ANN) structure



Forward Propagation



desired results

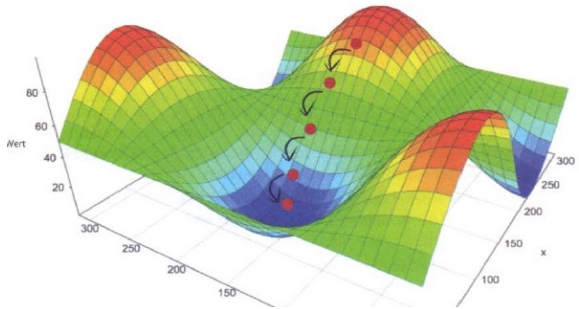
y_0

y_1

Error – difference between predicted output and desired output

$$\text{Error/Loss} = \sum_{j=0}^{n_L-1} \left(a_j{}^{(L)} - y_j\right)^2$$

Gradient of error is calculated with respect to each weight and bias

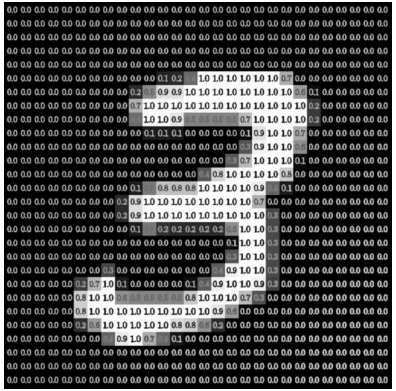


Adjust weights and biases via backpropagation

Back Propagation

Fully connected neural network

pixel representation of an image (grayscale image)



28 pixels

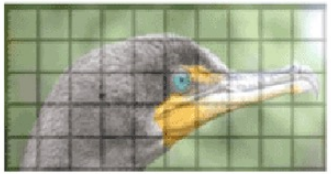
28 pixels

Flatten

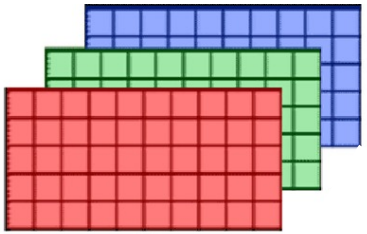
784 input Pixels

0
1
2
3
4
5
6
7
8
9

Color image



RGB



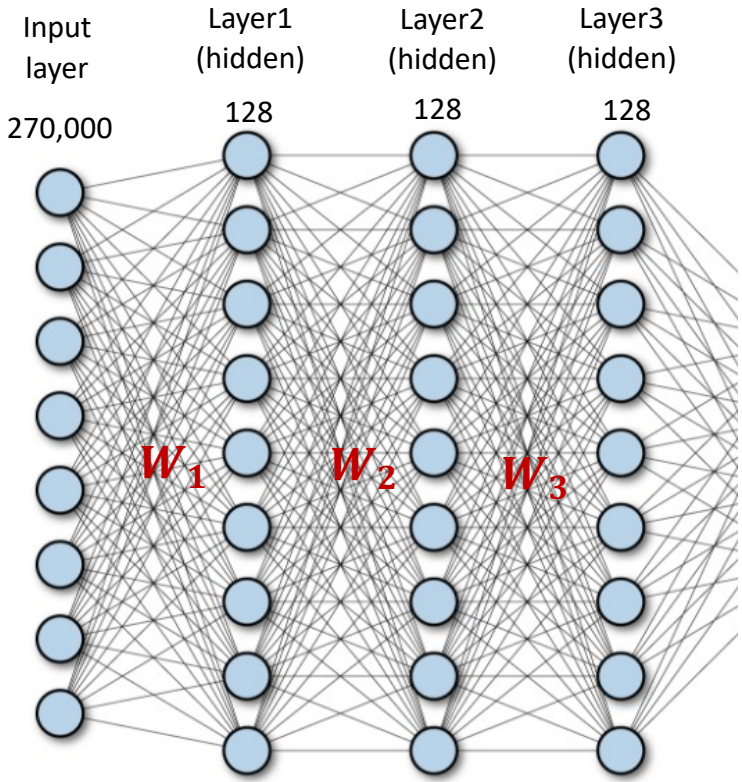


300x300x3 pixels

270,000 vector of pixel values

Flatten

Fully connected NN

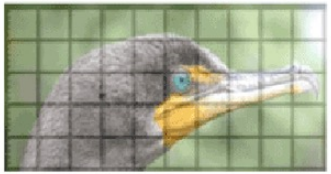


$\#W_1$: 270000x128=34,560,000 weights

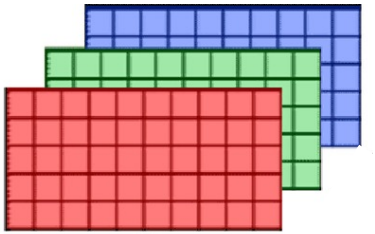
$\#W_2$: 34,560,000x128 ~4,4 Billion

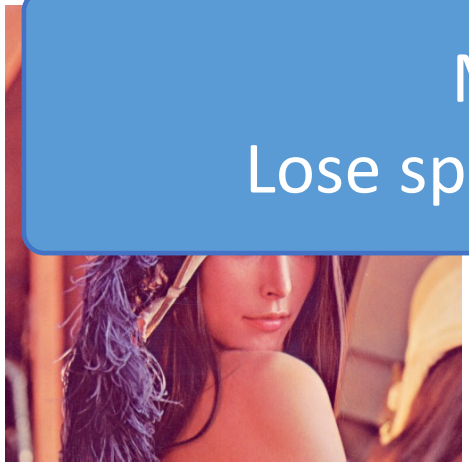
$\#W_3$: ~4,4x128 > 560 Billion

Color image



RGB

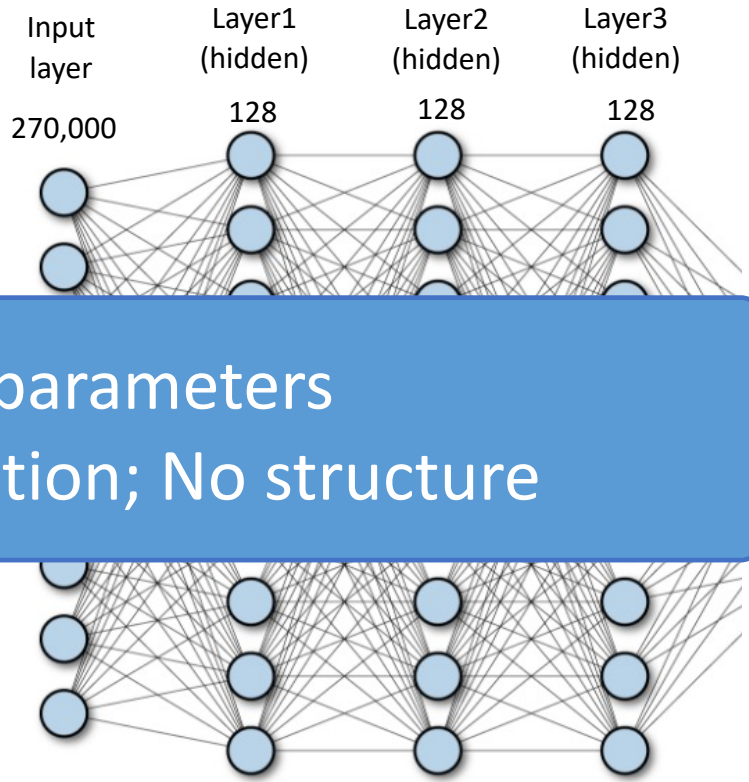




300x300x3 pixels

270,000 vector of pixel values

Fully connected NN



Many, many parameters

Lose spatial information; No structure

$\#W_1$: 270000x128=34,560,000 weights

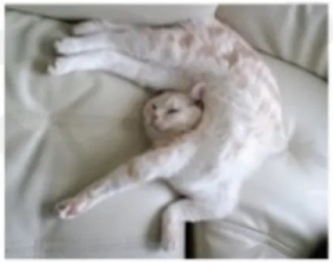
$\#W_2$: 34,560,000x128 ~4,4 Billion

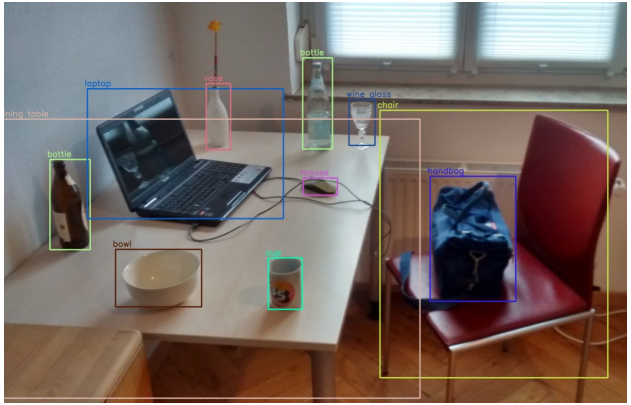
$\#W_3$: ~4,4x128 > 560 Billion

Viewpoint variations



Deformations





Illumination conditions



Intra-class variations





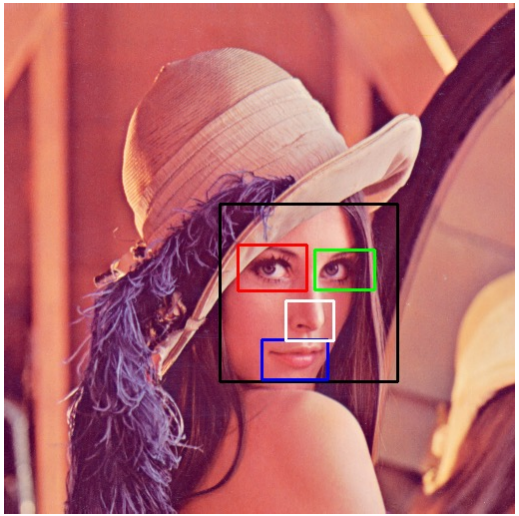
Manual Feature Extraction

http://introtodeeplearning.com

Better Way than FNN ?

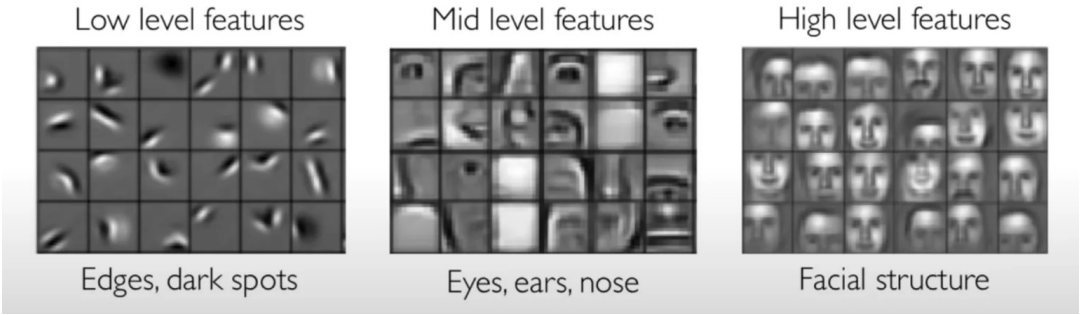
How can we use spatial structure in the input to inform the network?

Can we learn features directly from the data instead of hand engineering?



We want:

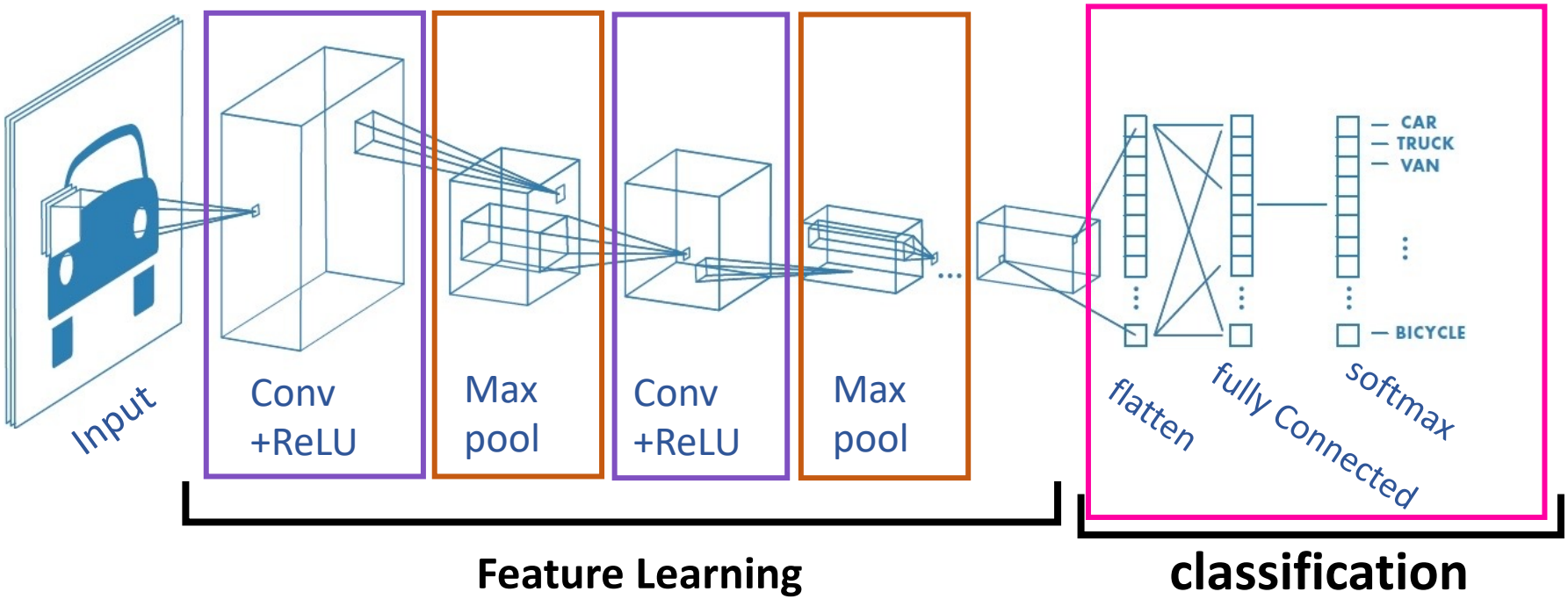
- Spatial locality
- Translation invariance
- Weight sharing (using the same weights for different parts)



Convolutional Neural Networks

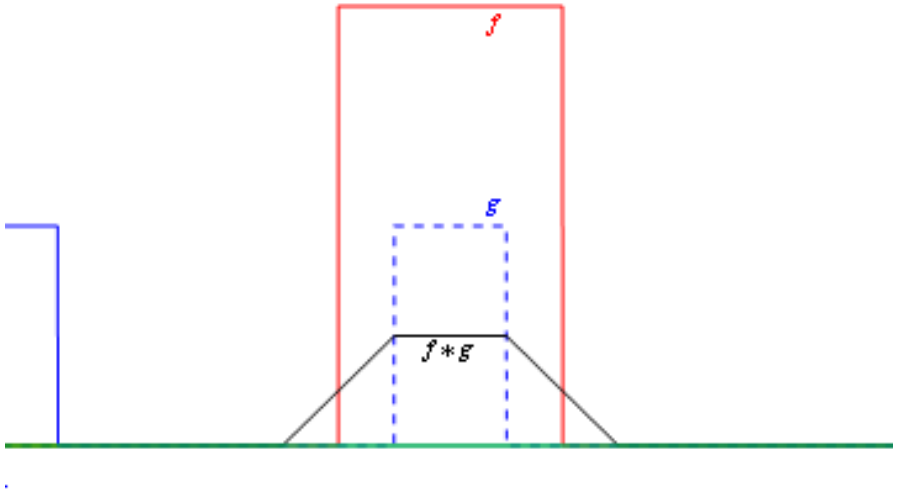
A ConvNet is a neural network that uses convolutional layers with shared, learnable filters to extract local features and build hierarchical representations from data.

CNN typical architecture:

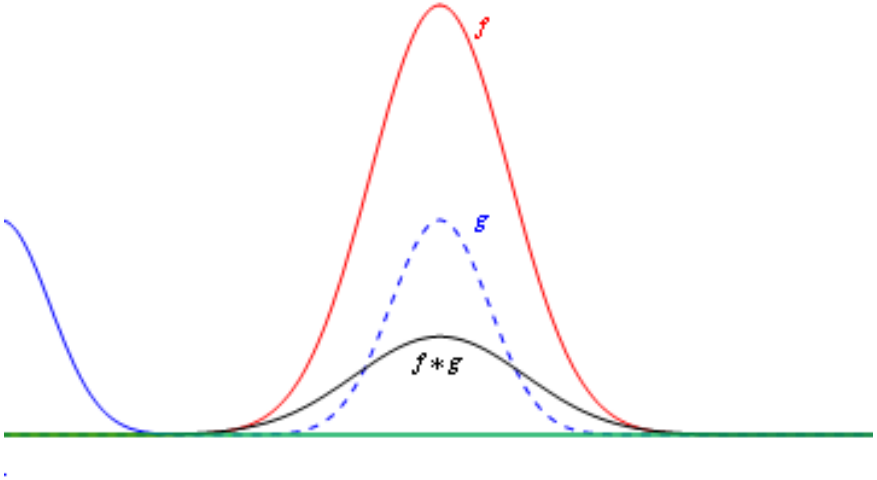


What are Convolutions?

$$(f * g)(t) := \int_{-\infty}^{\infty} f(\tau) g(t - \tau)\, d\tau$$



Convolution of two box functions



Convolution of two Gaussians

Application of a filter to a function

The ‘smaller’ one is typically called the **filter kernel**

Discrete convolution

f

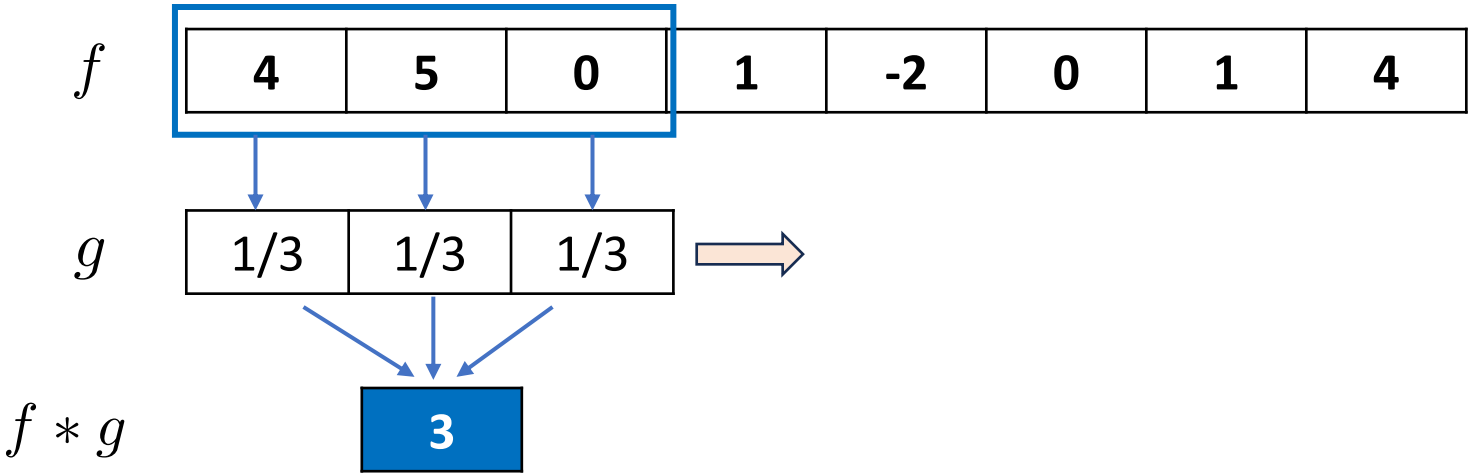
4	5	0	1	-2	0	1	4

g

1/3	1/3	1/3

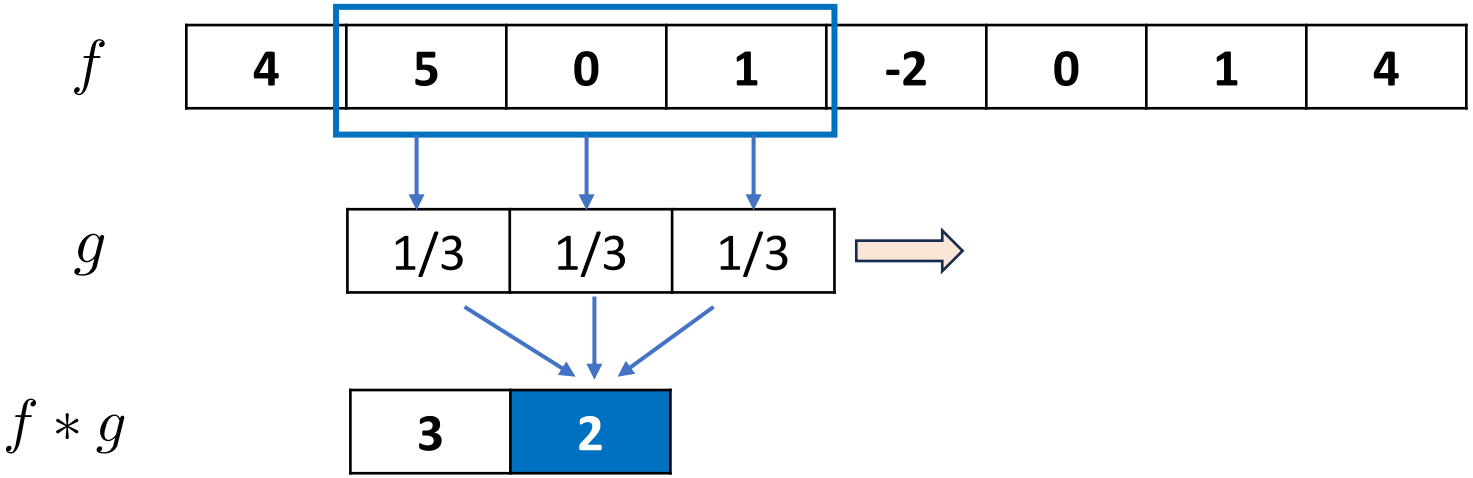
‘Slide’ filter kernel from left to right;
at each position, compute a single value in the output data

Discrete convolution



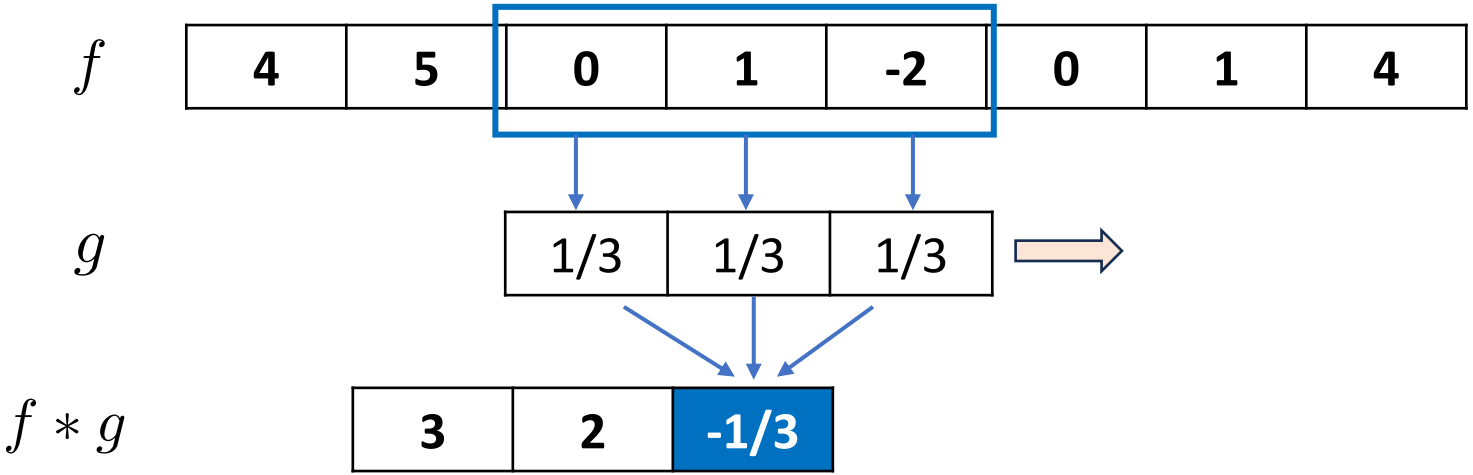
$$4 * \frac{1}{3} + 5 * \frac{1}{3} + 0 * \frac{1}{3} = 3$$

Discrete convolution



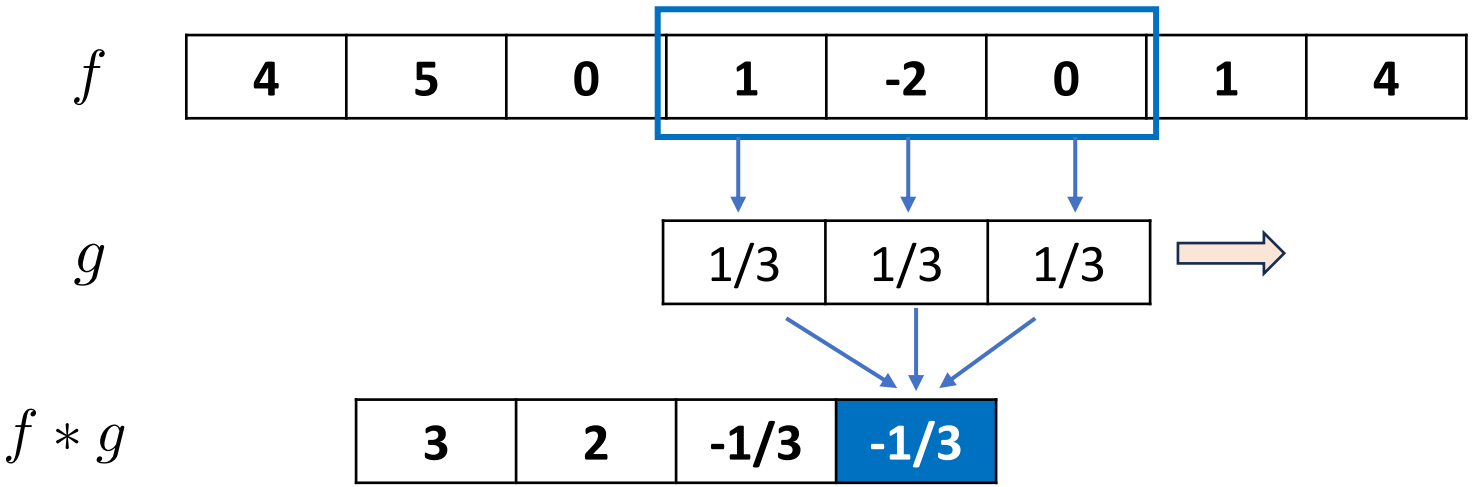
$$5 * \frac{1}{3} + 0 * \frac{1}{3} + 1 * \frac{1}{3} = 2$$

Discrete convolution



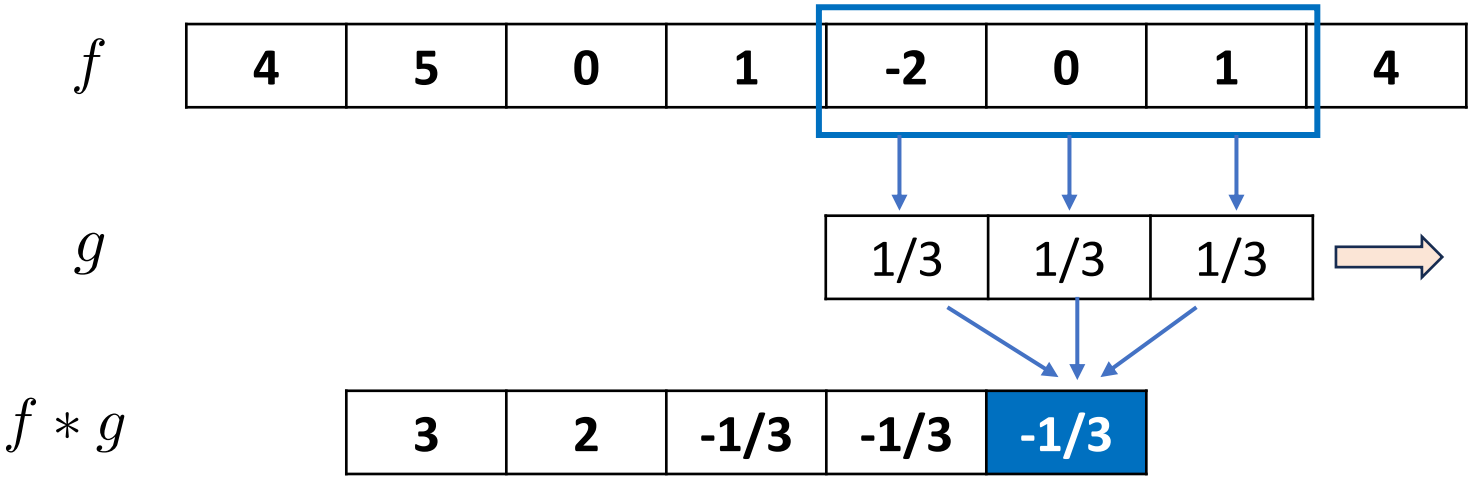
$$0 * \frac{1}{3} + 1 * \frac{1}{3} - 2 * \frac{1}{3} = -\frac{1}{3}$$

Discrete convolution



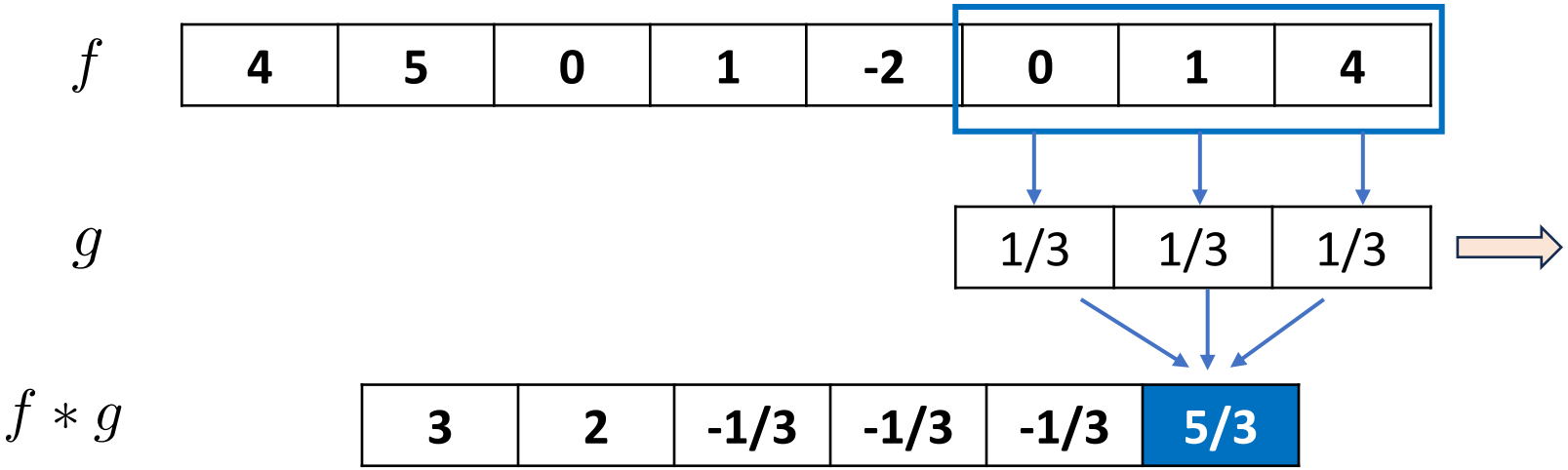
$$1 * \frac{1}{3} - 2 * \frac{1}{3} + 0 * \frac{1}{3} = -\frac{1}{3}$$

Discrete convolution



$$-2 * \frac{1}{3} + 0 * \frac{1}{3} + 1 * \frac{1}{3} = -\frac{1}{3}$$

Discrete convolution



$$0 * \frac{1}{3} + 1 * \frac{1}{3} + 4 * \frac{1}{3} = \frac{5}{3}$$

Discrete convolution

What if we want that the Output has the same size?

f

4	5	0	1	-2	0	1	4

g

1/3	1/3	1/3

$f * g$

3	2	-1/3	-1/3	-1/3	5/3

Discrete convolution

What if we want that the Output has the same size?

Pad (often 0's)

f

0	4	5	0	1	-2	0	1	4	0

g

1/3	1/3	1/3

$f * g$

?	3	2	-1/3	-1/3	-1/3	5/3	?

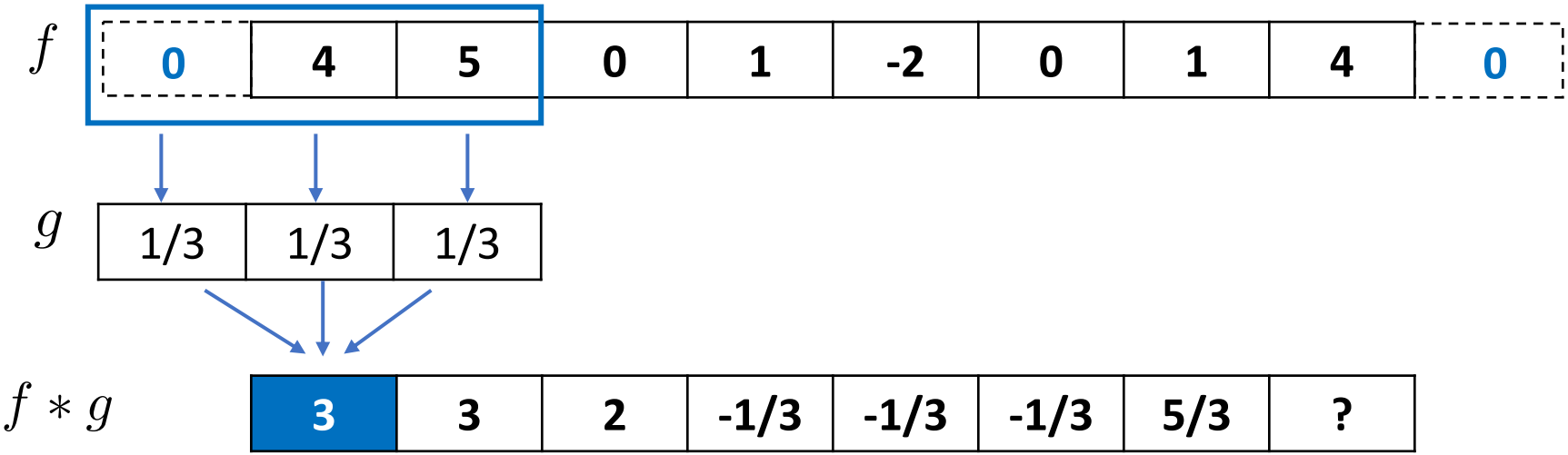
Padding:

It helps to preserve the input spatial dimension by adding extra pixels around the input borders

Discrete convolution

Output has the same size?

Pad (often 0's)

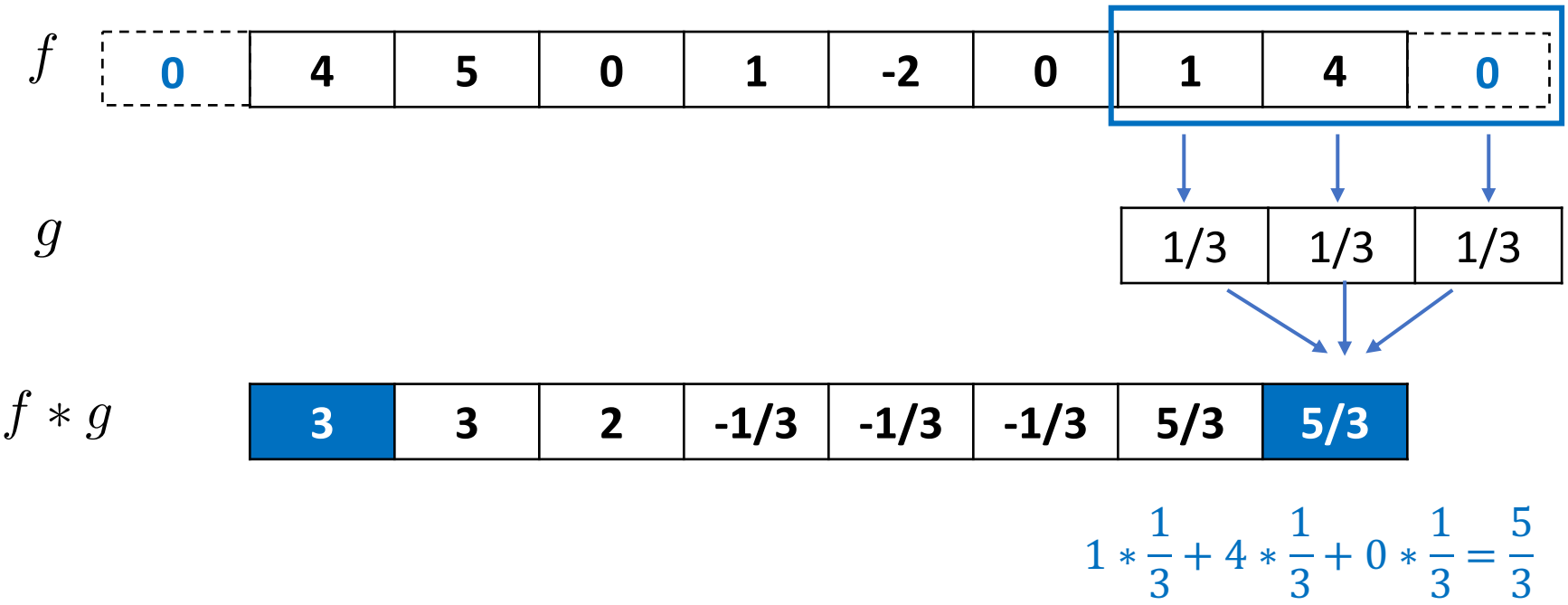


$$0 * \frac{1}{3} + 4 * \frac{1}{3} + 5 * \frac{1}{3} = 3$$

Discrete convolution

Output has the same size?

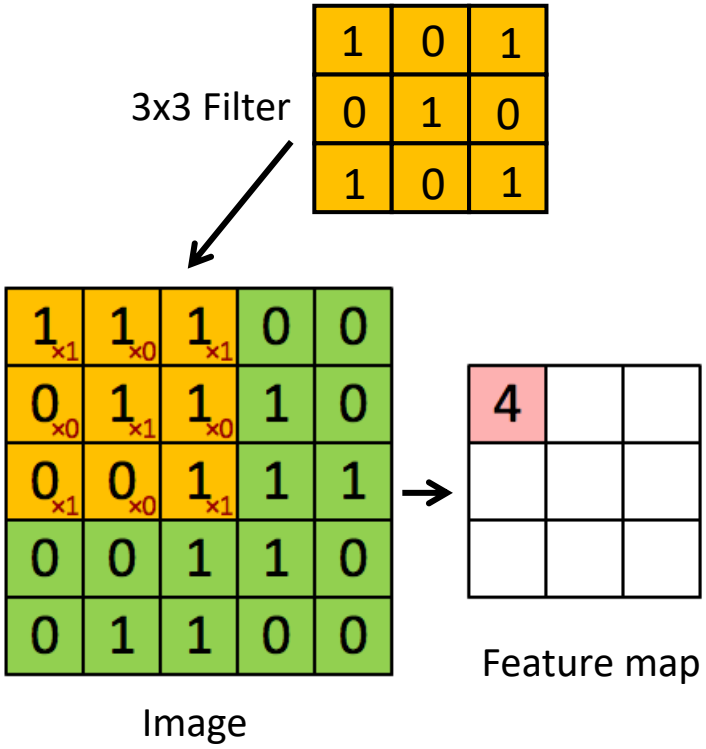
Pad (often 0's)



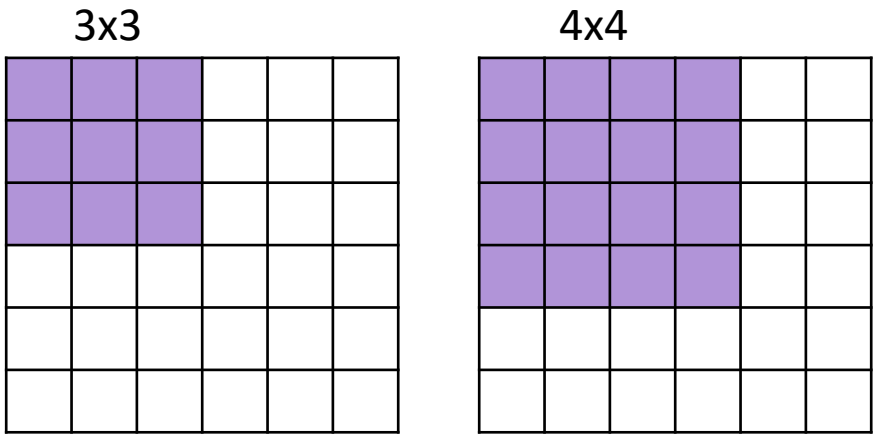
$$1 * \frac{1}{3} + 4 * \frac{1}{3} + 0 * \frac{1}{3} = \frac{5}{3}$$

Discrete convolution in 2D

Convolutions on Images



Kernel size is the height and width of the convolution filter applied to the input



Kernel/Filter (imaging processing)

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Original

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

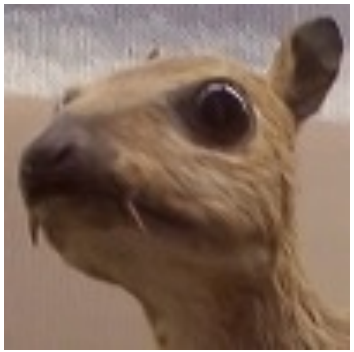
Sharpen

$$\frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Box blur

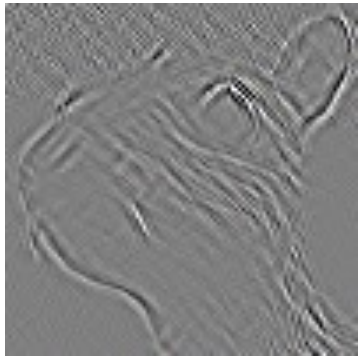
$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Edge Detect







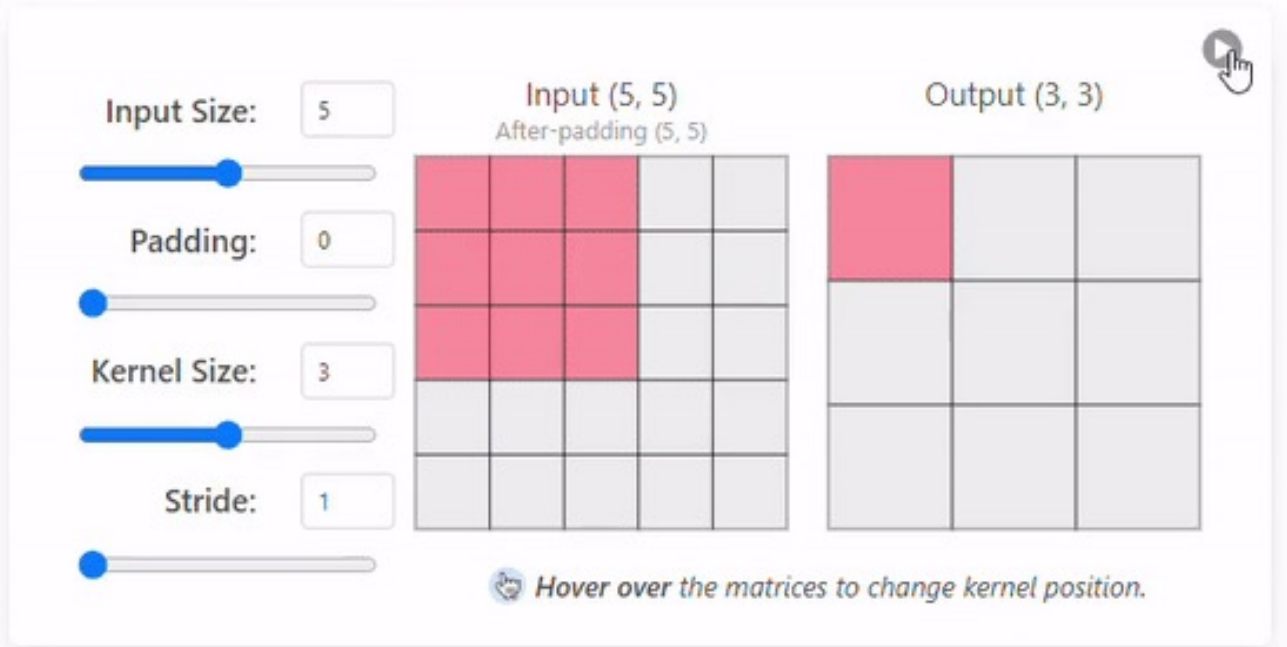


https://setosa.io/ev/image-kernels/

Discrete convolution in 2D

Stride: Filter step size that determines how many pixels the kernel shifts over the input at a time

Stride = 1

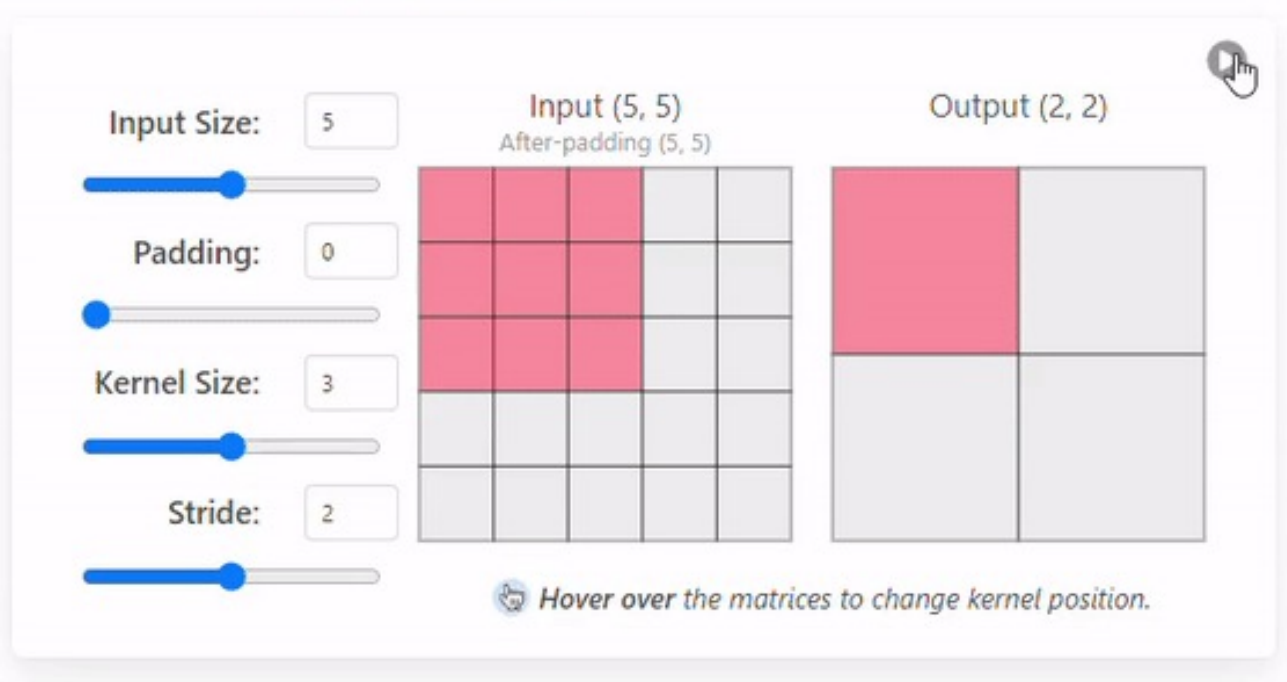


https://poloclub.github.io/cnn-explainer/

Discrete convolution in 2D

Stride: Filter step size that determines how many pixels the kernel shifts over the input at a time

Stride = 2

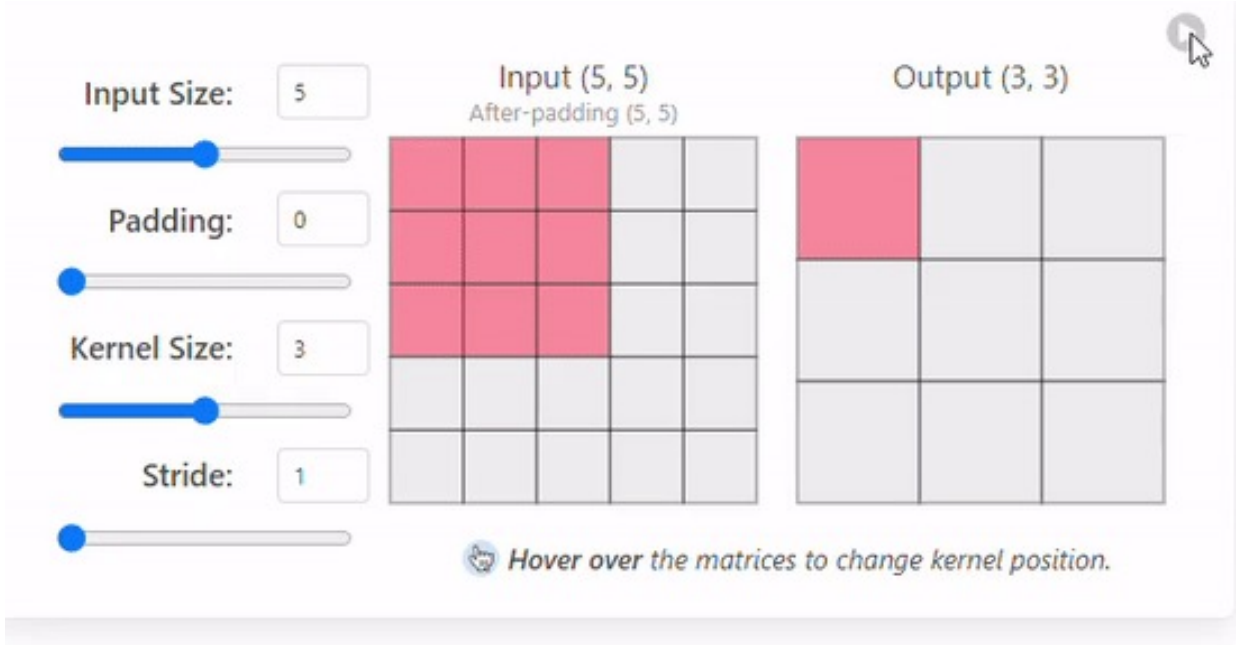


https://poloclub.github.io/cnn-explainer/

Discrete convolution in 2D

Padding to preserve the input spatial dimension by adding extra pixels around the input image borders

Padding = 0

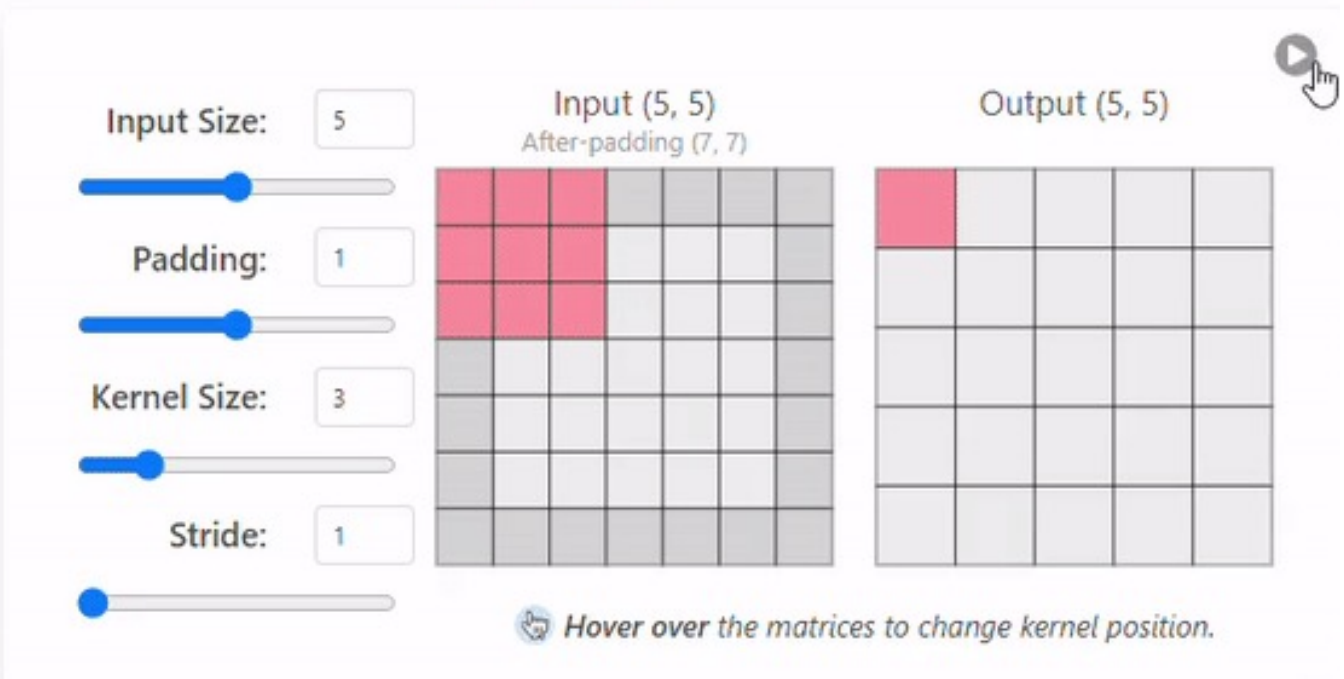


https://poloclub.github.io/cnn-explainer/

Discrete convolution in 2D

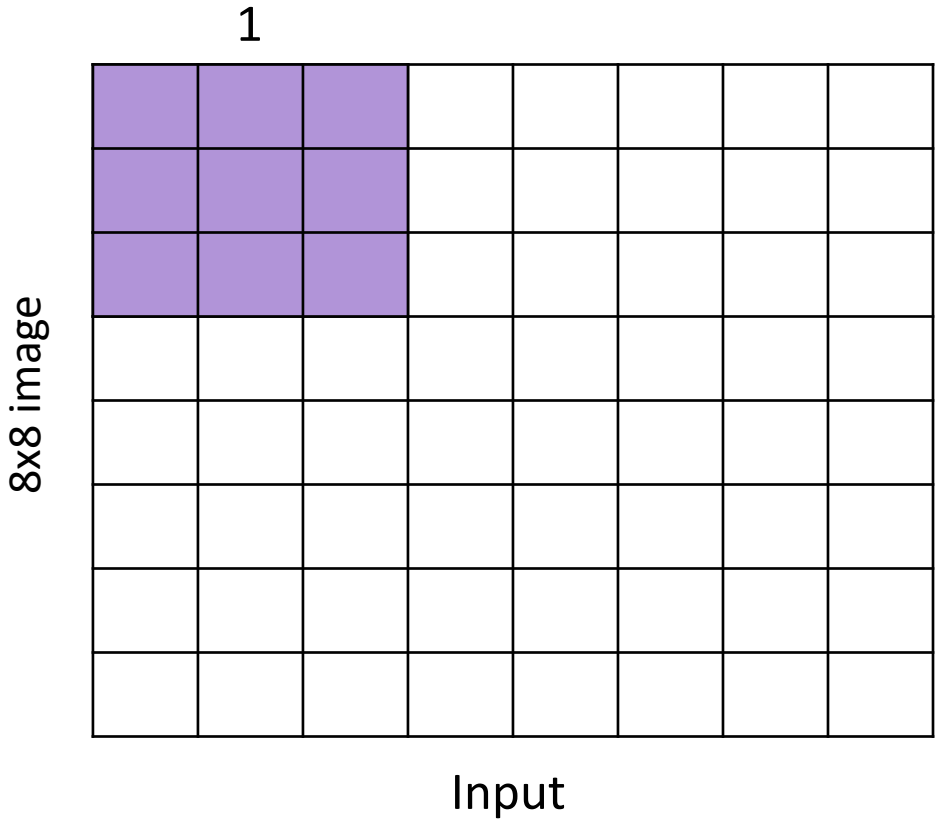
Padding to preserve the input spatial dimension by adding extra pixels around the input image borders

Padding = 1



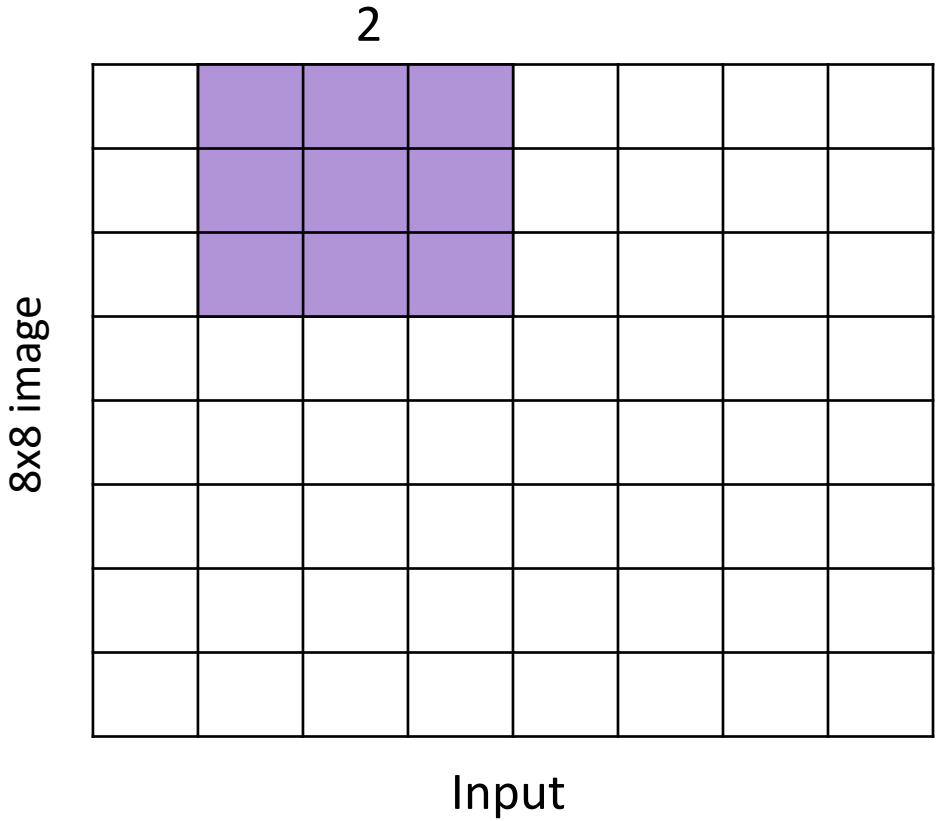
https://poloclub.github.io/cnn-explainer/

How big is the output?



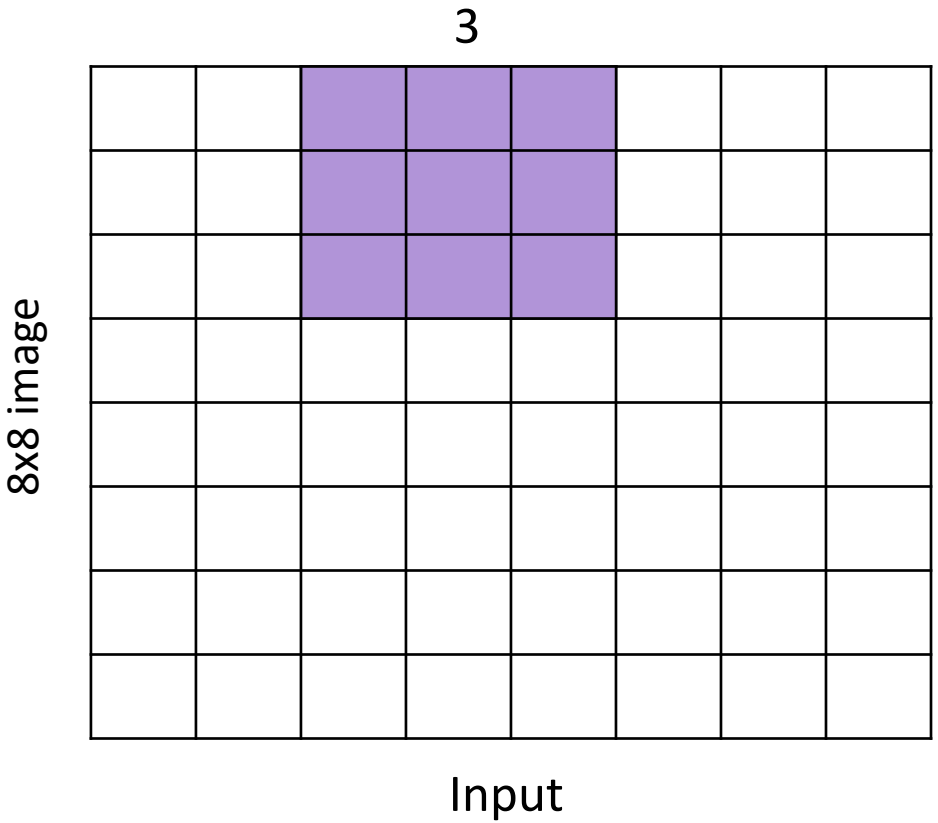
Input: 8x8
Filter: 3x3
Stride: 1

How big is the output?



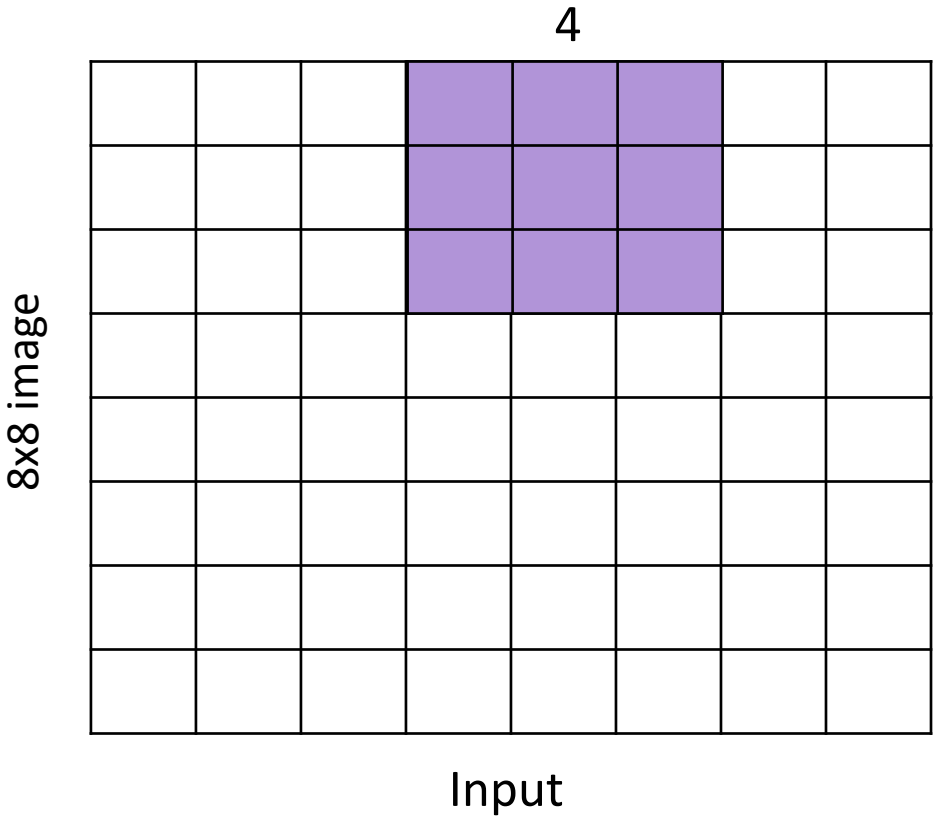
Input: 8x8
Filter: 3x3
Stride: 1

How big is the output?



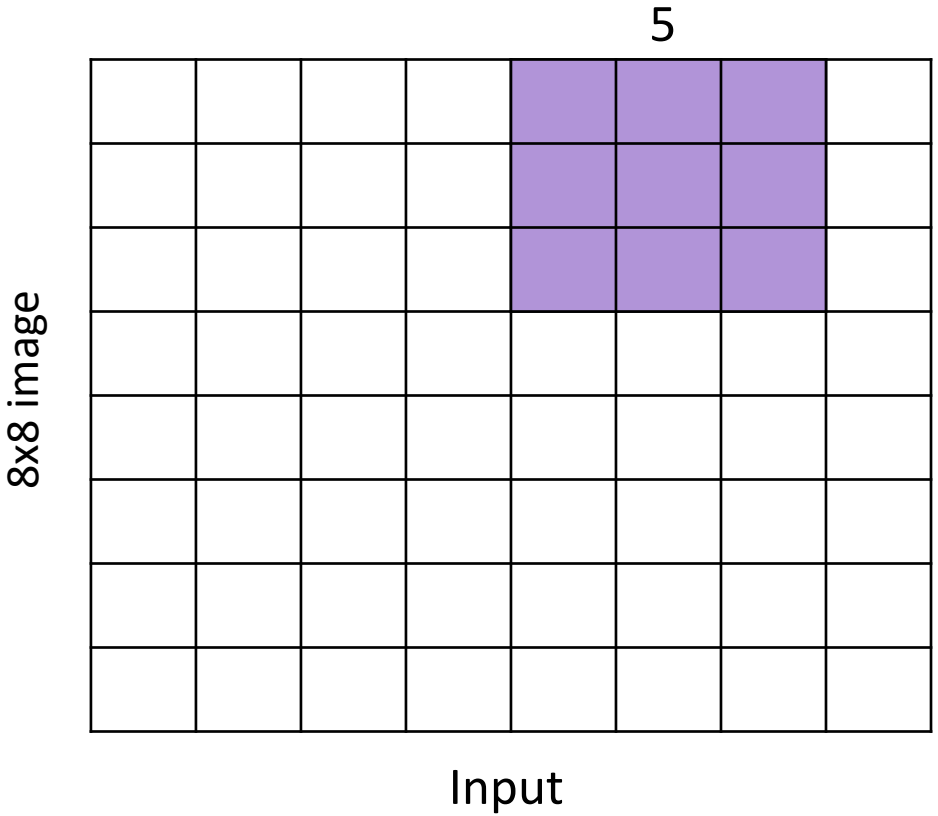
Input: 8x8
Filter: 3x3
Stride: 1

How big is the output?



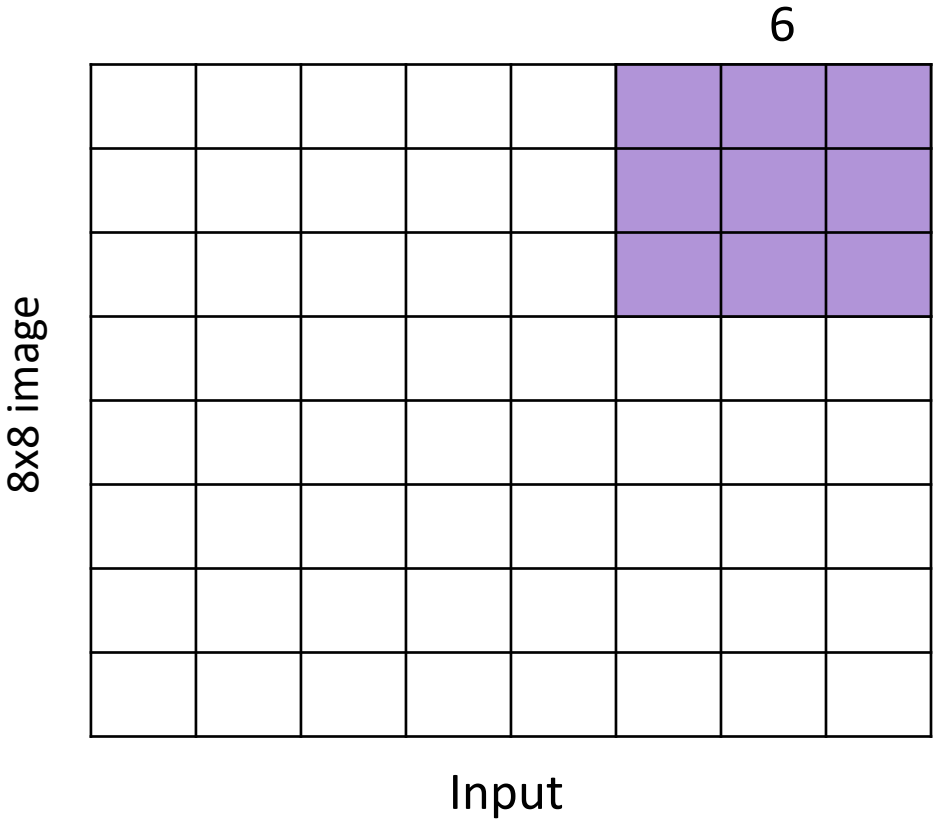
Input: 8x8
Filter: 3x3
Stride: 1

How big is the output?



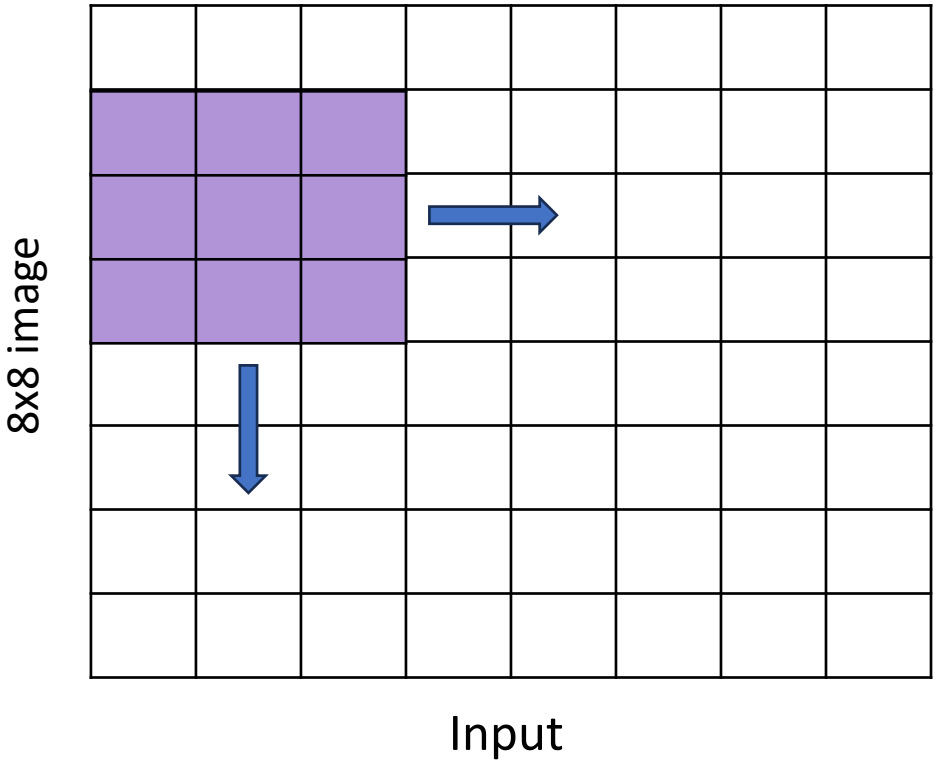
Input: 8x8
Filter: 3x3
Stride: 1

How big is the output?

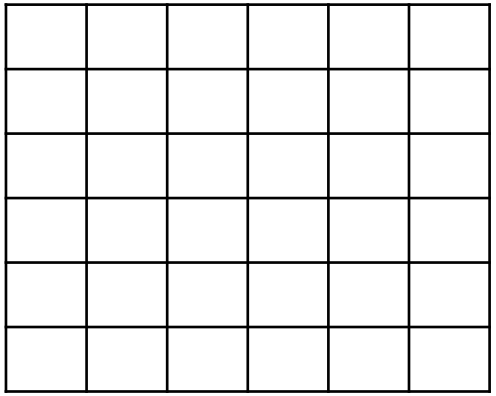


Input: 8x8
Filter: 3x3
Stride: 1

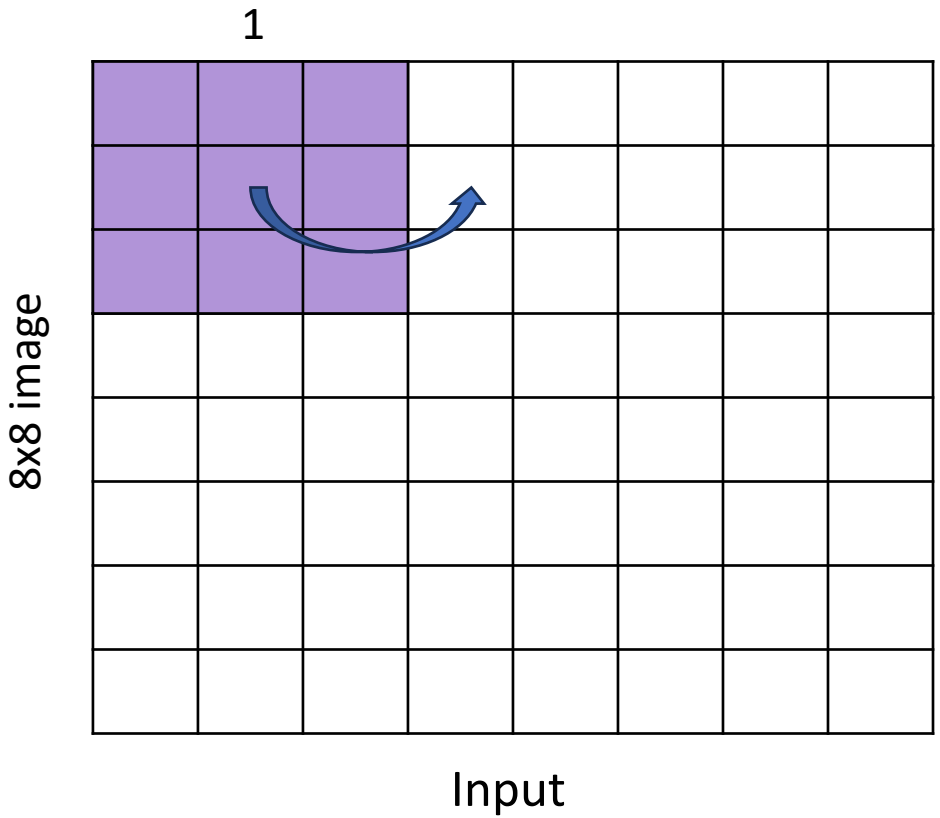
How big is the output?



Input: 8x8
Filter: 3x3
Stride: 1
Output: 6x6

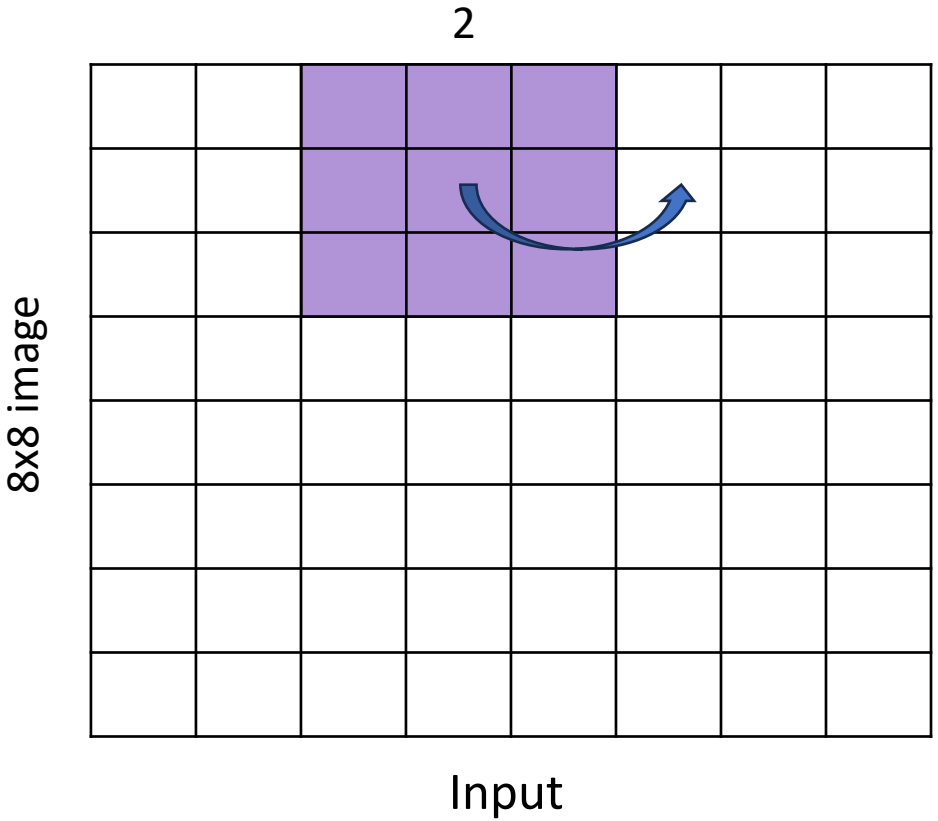


How big is the output?



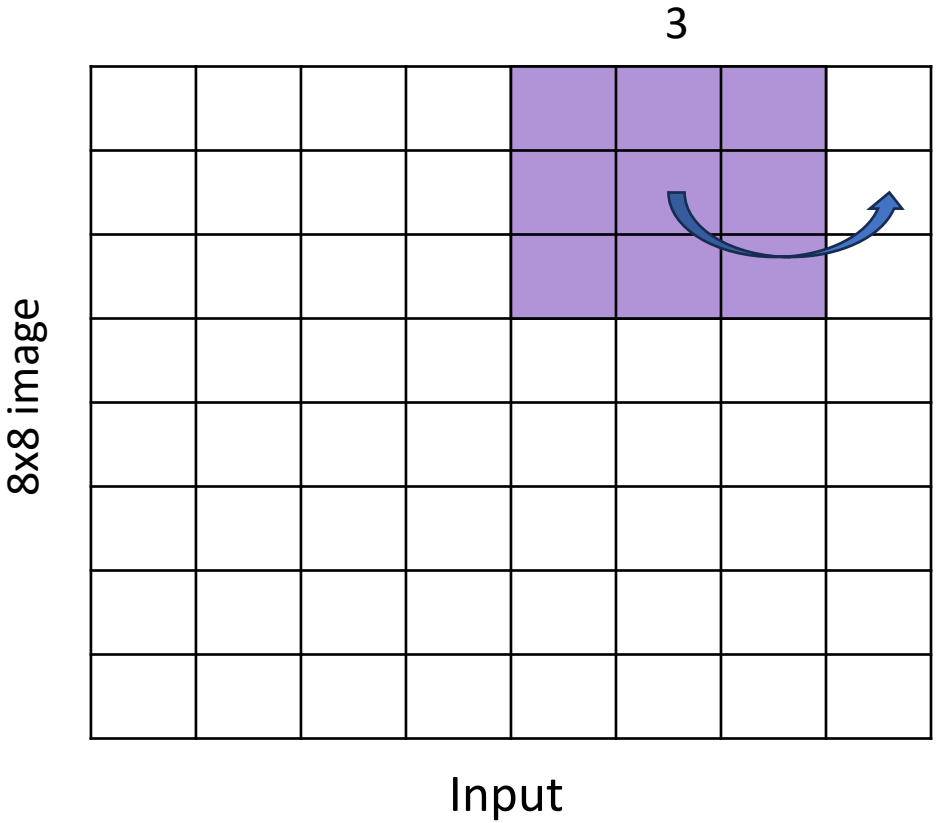
Input: 8x8
Filter: 3x3
Stride: 2

How big is the output?



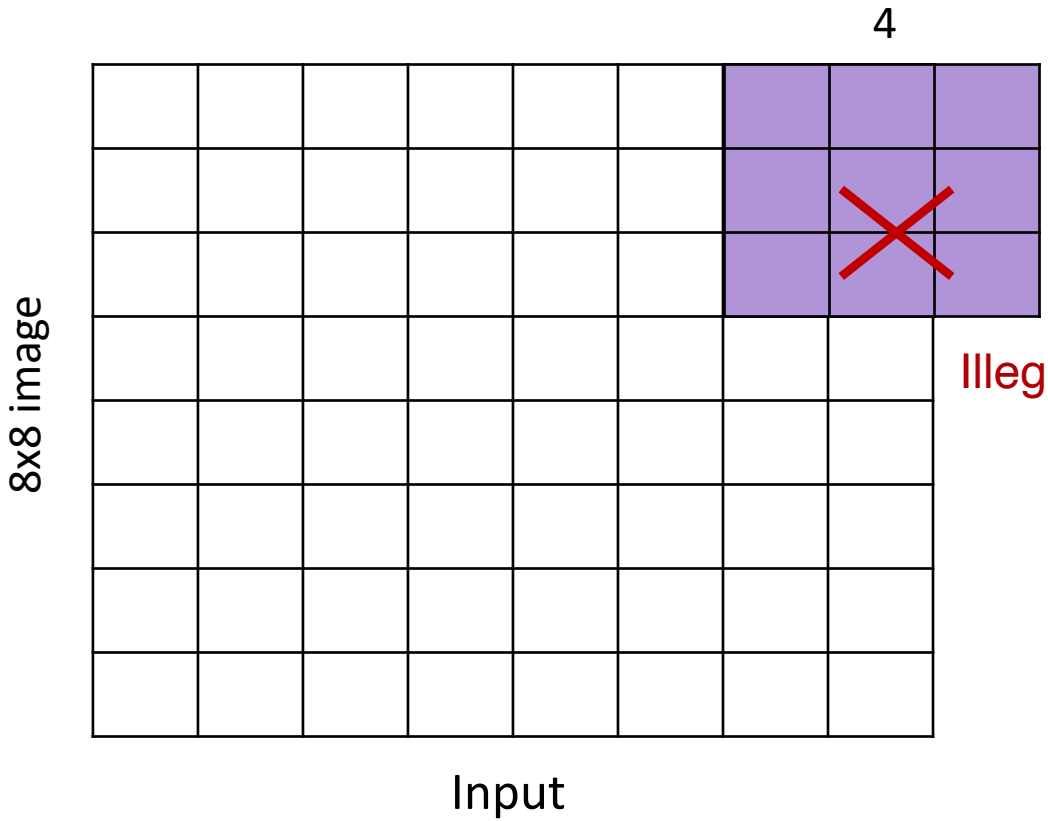
Input: 8x8
Filter: 3x3
Stride: 2

How big is the output?



Input: 8x8
Filter: 3x3
Stride: 2

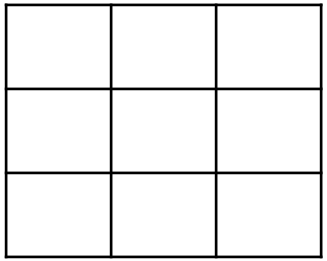
How big is the output?



Input: 8x8
Filter: 3x3
Stride: 2

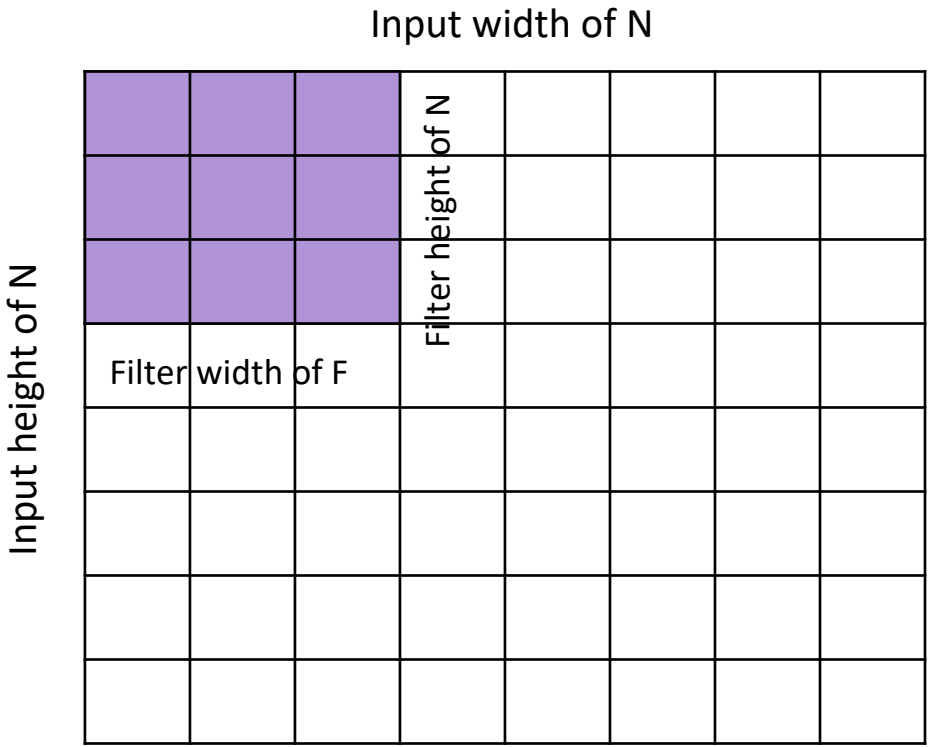
Illegal stride for input & filter size!

With certain strides, we may not be able to cover all of the input



Output: 3x3

How big is the output?



Input: $N \times N$

Filter: $F \times F$

Stride: S

Output: $\left(\frac{N-F}{S}+1\right) \times \left(\frac{N-F}{S}+1\right)$

$N = 8, F = 3, S = 1$: $\frac{8-3}{1}+1 = 6$

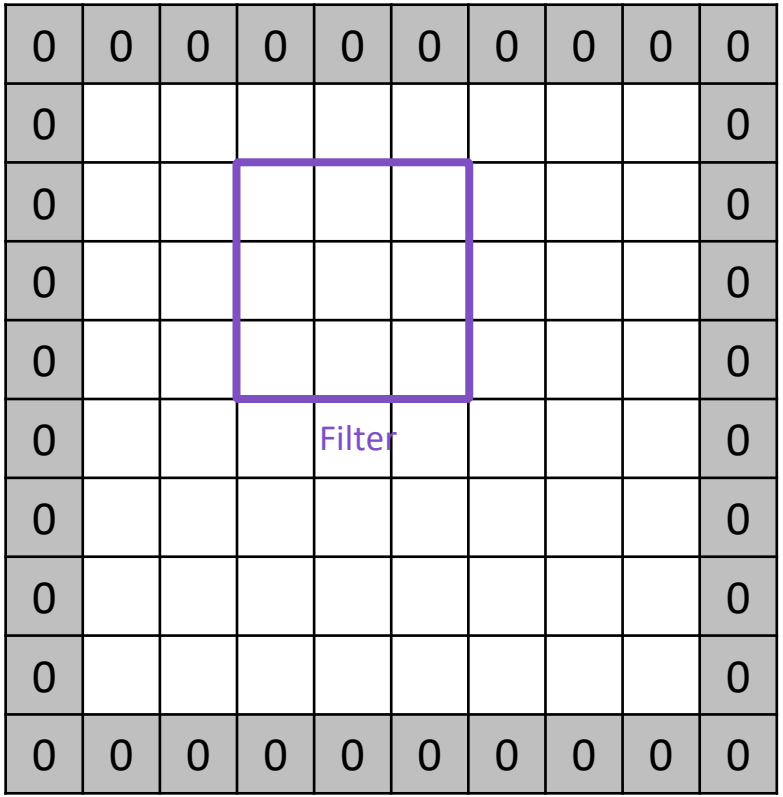
$N = 8, F = 3, S = 2$: $\frac{8-3}{2}+1 = 3.5$

Fractions are illegal

Sizes get small too quickly

Corner pixel is only used once

Padding



Most common is ‘zero’ padding

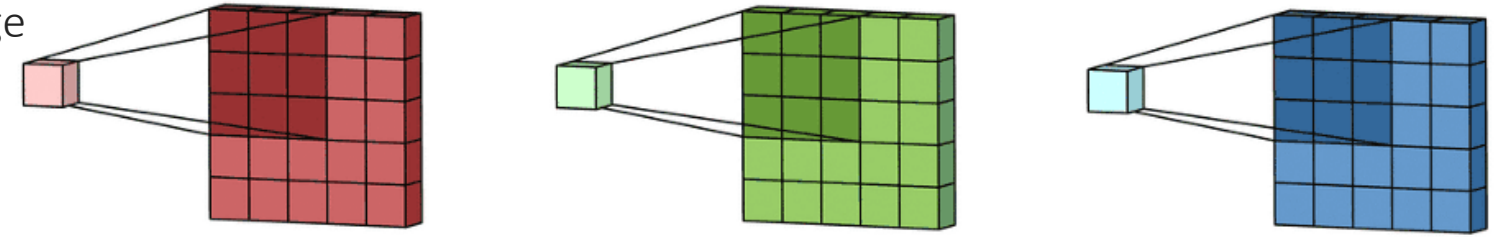
Input: $N \times N$
Filter: $F \times F$
Stride: S
Padding: P

Output Size:

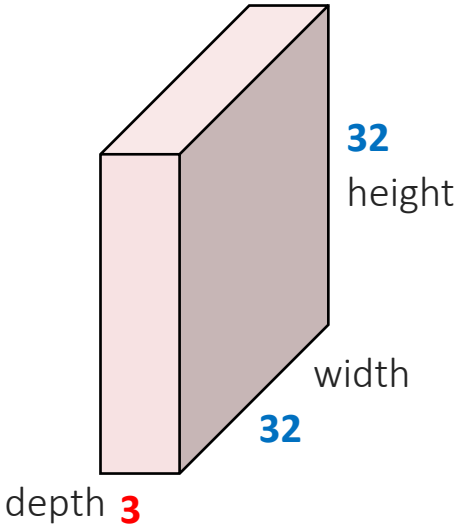
$$(\lfloor \frac{N + 2P - F}{S} \rfloor + 1) \times (\lfloor \frac{N + 2P - F}{S} \rfloor + 1)$$

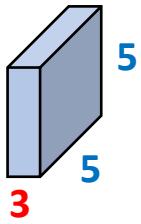
Convolutions on RGB Images

RGB image



separate kernels for each color channel





Filter 5x5x3

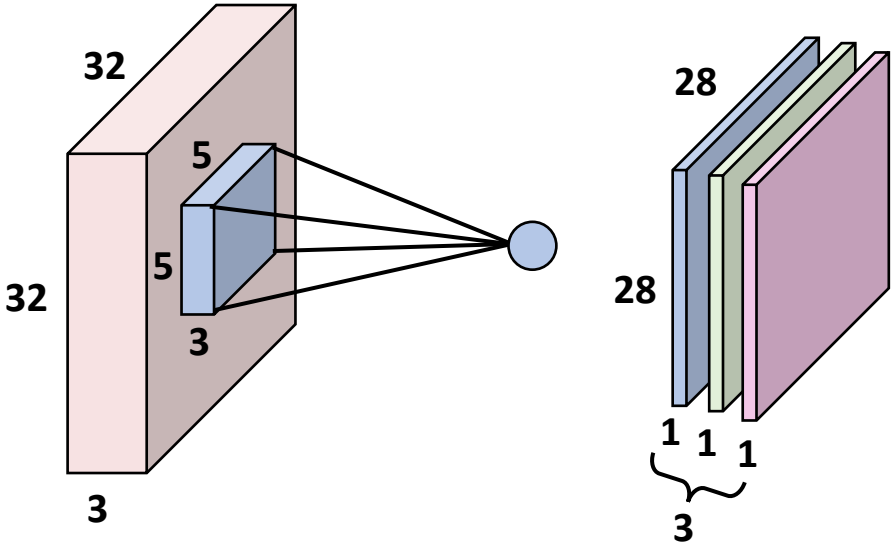
Depth dimension must match;
i.e., filter extends the full depth of the input

Depth can be a large number;
One can have as many filter as one wants

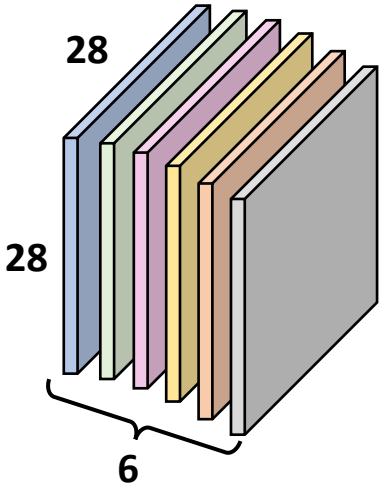
Convolve filter with image
i.e., 'slide' over the image spatially and apply filter at each location and compute the dot products

Convolution on Images

activation/feature maps



If we apply 6 5x5 filters, we have:

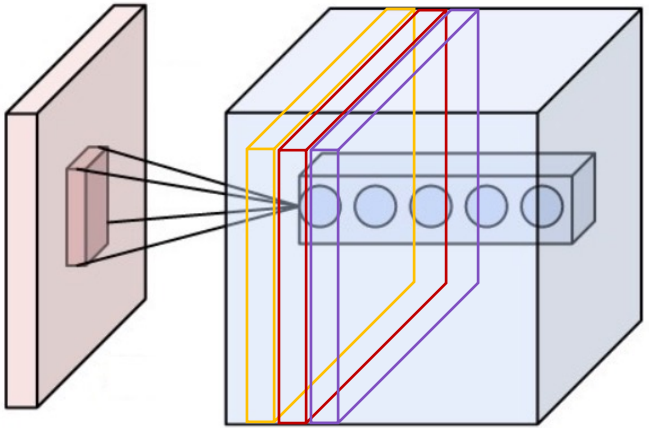


Convolve filter with image
i.e., 'slide' over the image spatially and apply filter at each location and compute the dot products

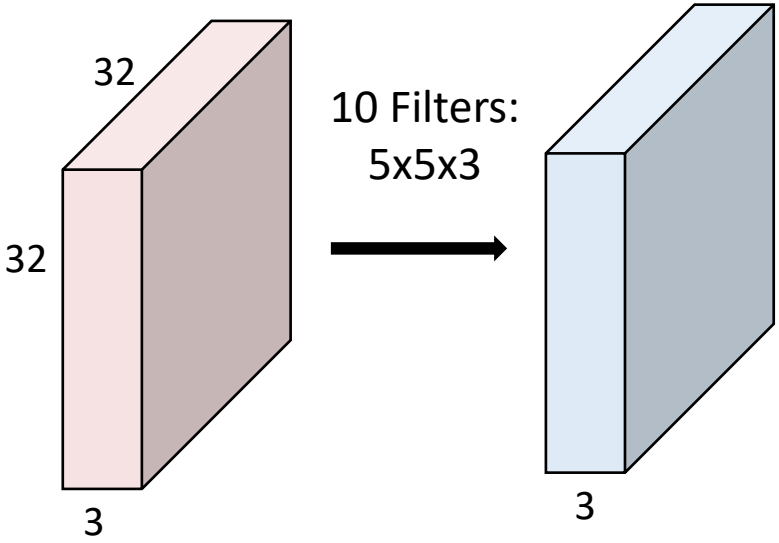
Each filter captures a different image characteristic

Dimensions of a Convolution Layer

Summary. To summarize, the Conv Layer:



- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

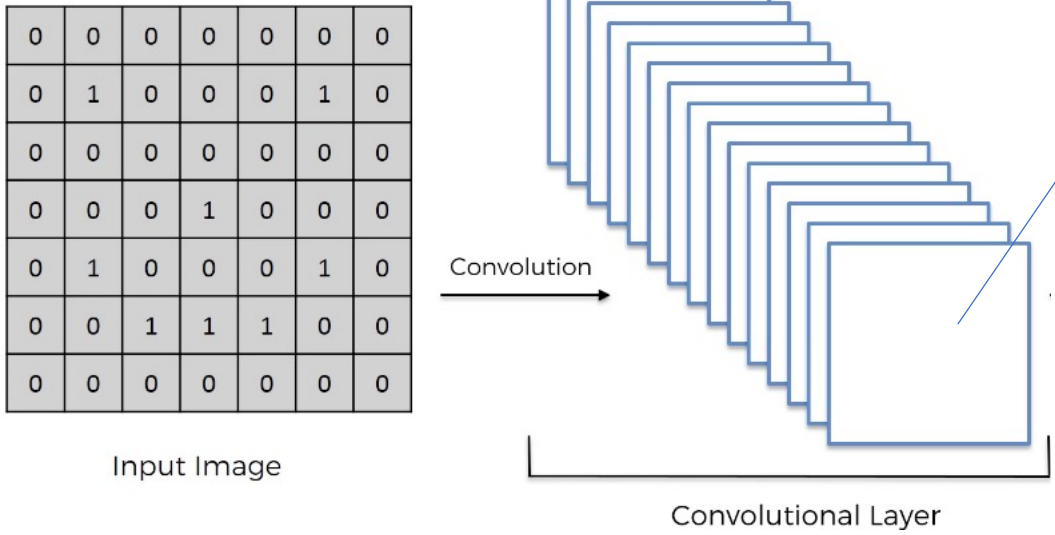


Input image: $32 \times 32 \times 3$
10 Filters: 5×5
Stride: 1
Padding: 2

Output Size is: 32 $(\lfloor \frac{N + 2P - F}{S} \rfloor + 1)$
i.e., $32 \times 32 \times 10$

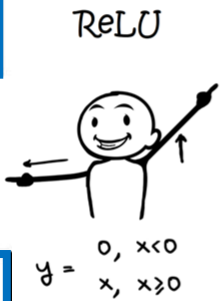
Number of parameters (weights):
Each filter has 5×5×3 + 1 = 76 params (**+1** for bias)
We have 76 · 10 = 760 parameters in layer

After Convolution



1	12	4
-79	134	-345
-3	-5	14

ReLU

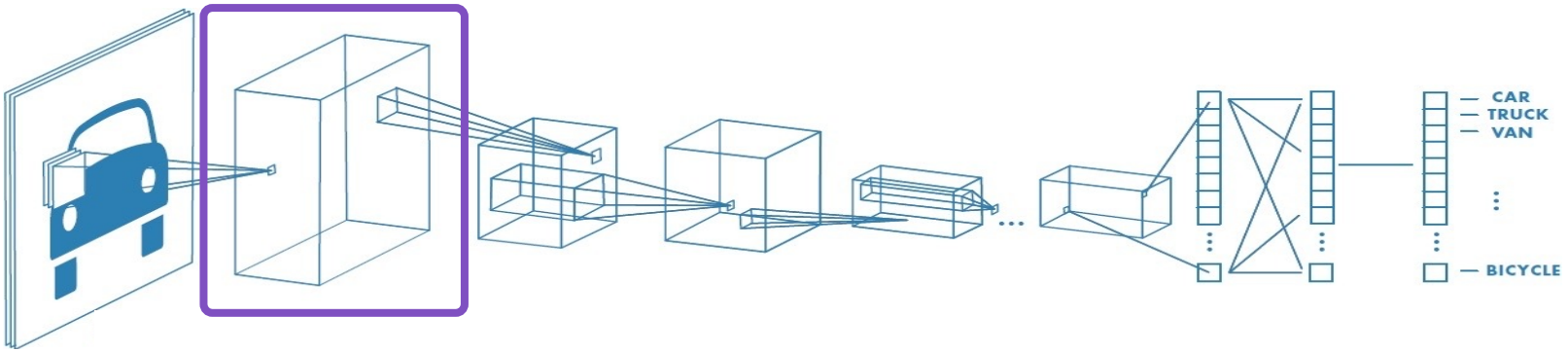


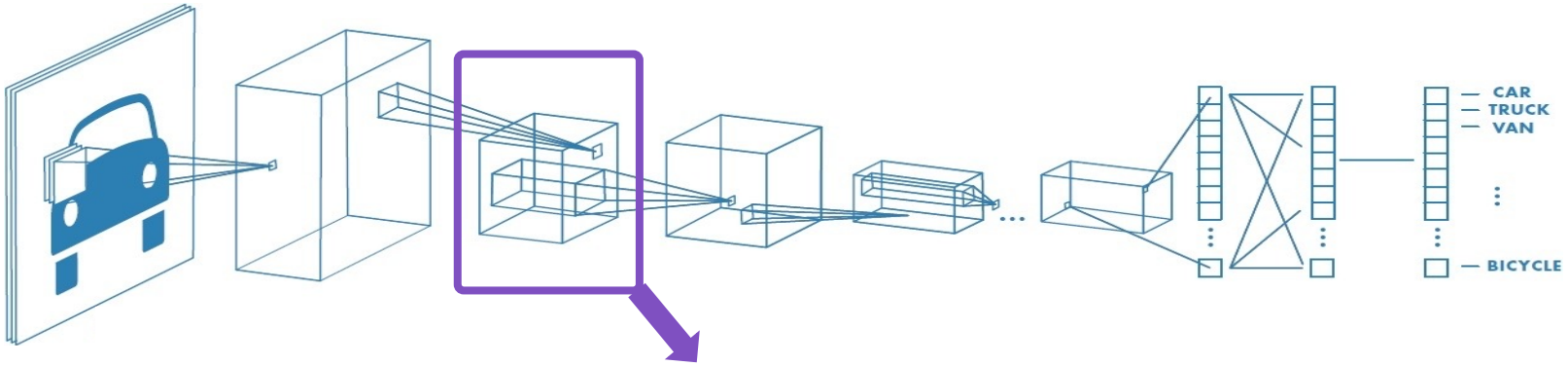
1	12	0
0	134	0
0	0	14

After obtaining feature map, apply an elementwise non-linearity to obtain a transformed feature map (same size)

Convolutional layer

- Small number of parameters to learn compared to a fully connected layer
- Preserves spatial structure (output of a convolutional layer is shaped like an image)
- Translation equivariant: passing a translated image through a convolutional layer is (almost) equivalent to translating the convolution output (but be careful of image boundaries)





Pooling Layers

downsampling operation

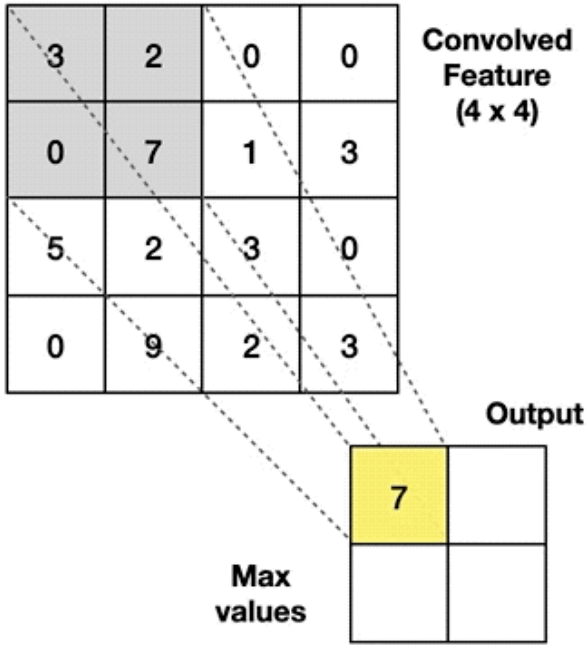
For most ConvNets, convolution is often followed by pooling:

- Create a smaller representation while retaining the most important information
- The “Max Pooling” operation is the most common one

Max Pooling

Take the **highest** value from the area covered by the kernel

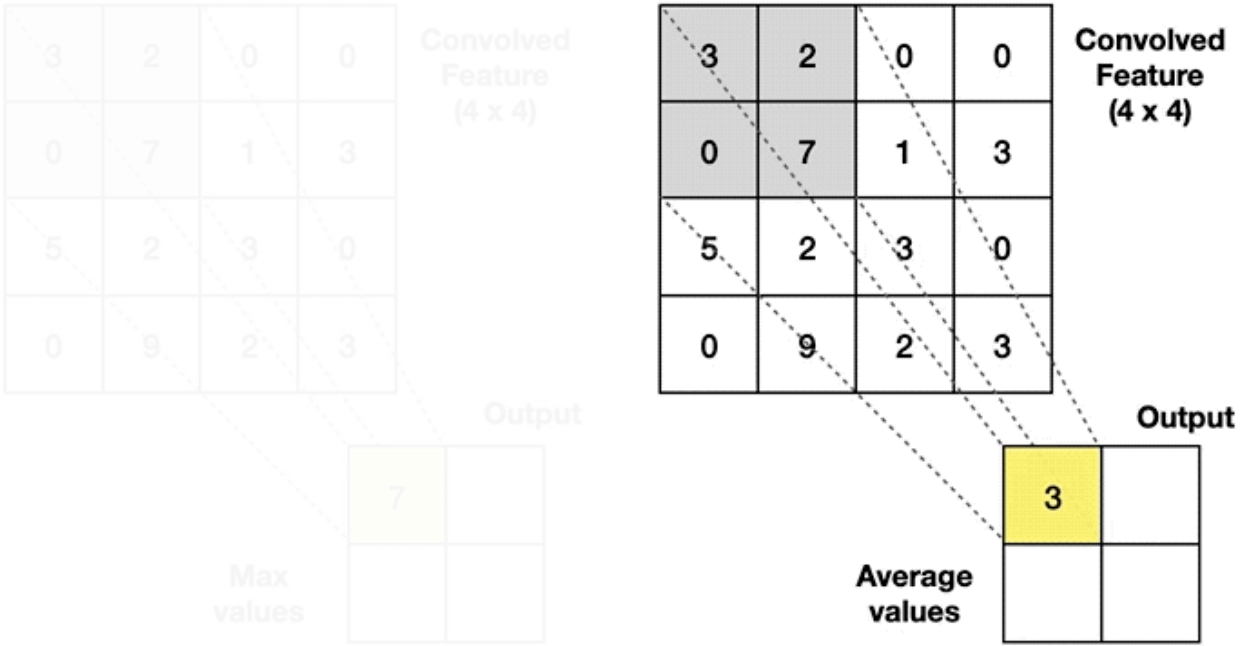
Example: Kernel of size 2 x 2; stride=(2,2)



Average Pooling

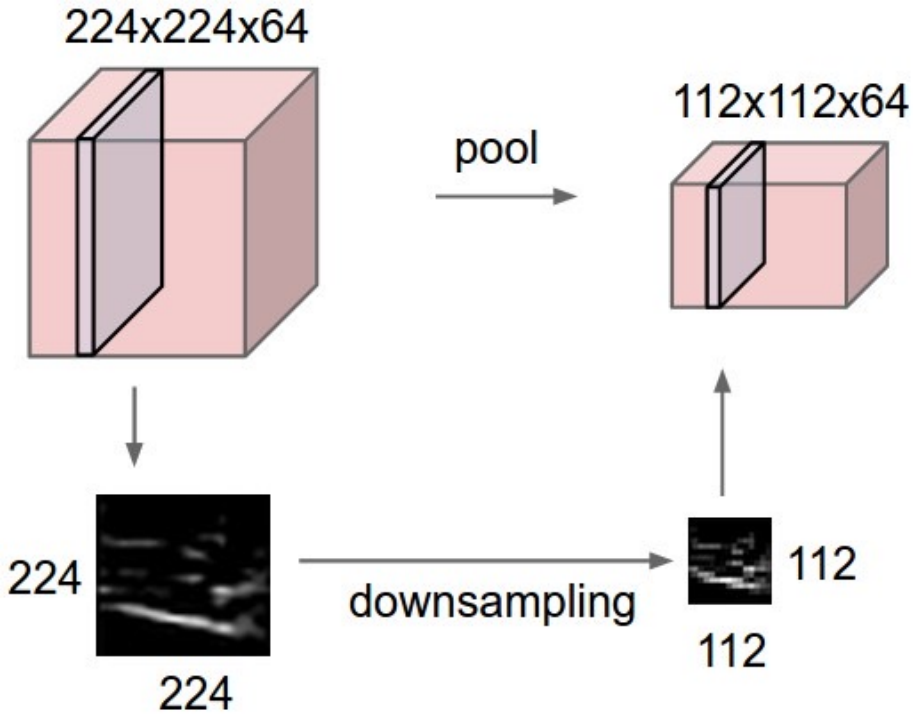
Calculate the **average** value from the area covered by the kernel

Example: Kernel of size 2 x 2; stride=(2,2)



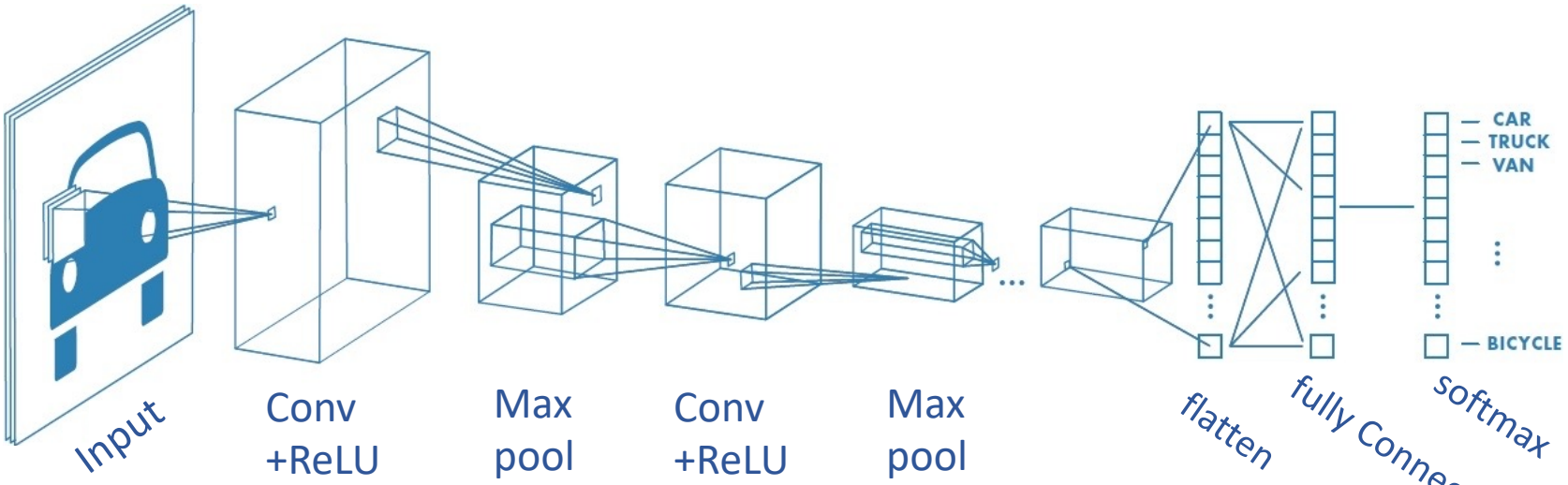
Pooling Layers

It downsamples the volume spatially, independently in each depth slice of the input volume



Pooling Layers

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires two hyperparameters:
  - their spatial extent F,
  - the stride S,
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F)/S + 1$
  - $H_2 = (H_1 - F)/S + 1$
  - $D_2 = D_1$
- Introduces zero parameters since it computes a fixed function of the input
- For Pooling layers, it is not common to pad the input using zero-padding.



Feature Learning

classification

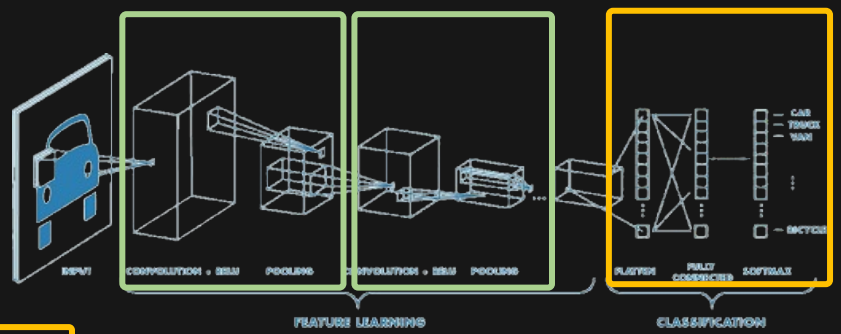
Object detection, Segmentation, Probabilistic control,…

```python
import tensorflow as tf

def generate_model():
    model = tf.keras.Sequential([
        # input: 28×28 grayscale image  -> shape (batch, 28, 28, 1)
        tf.keras.layers.Conv2D(32, (3, 3), activation="relu",
                               input_shape=(28, 28, 1)),
        tf.keras.layers.MaxPool2D(pool_size=(2, 2)),

        # second conv block
        tf.keras.layers.Conv2D(64, (3, 3), activation="relu"),
        tf.keras.layers.MaxPool2D(pool_size=(2, 2)),

        # fully connected classifier
        tf.keras.layers.Flatten()
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax")  # 10 classes
    ])
    return model
```

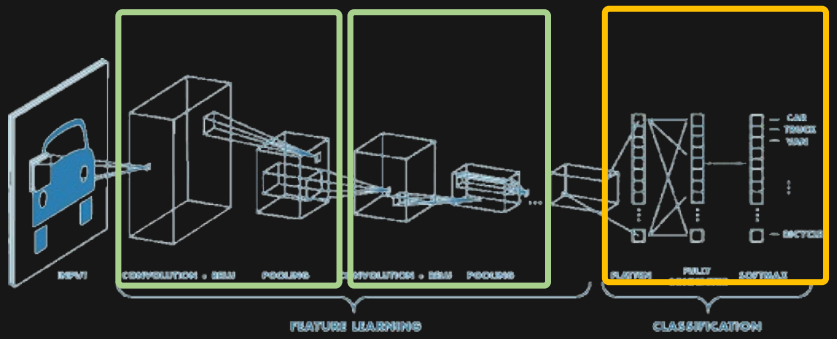


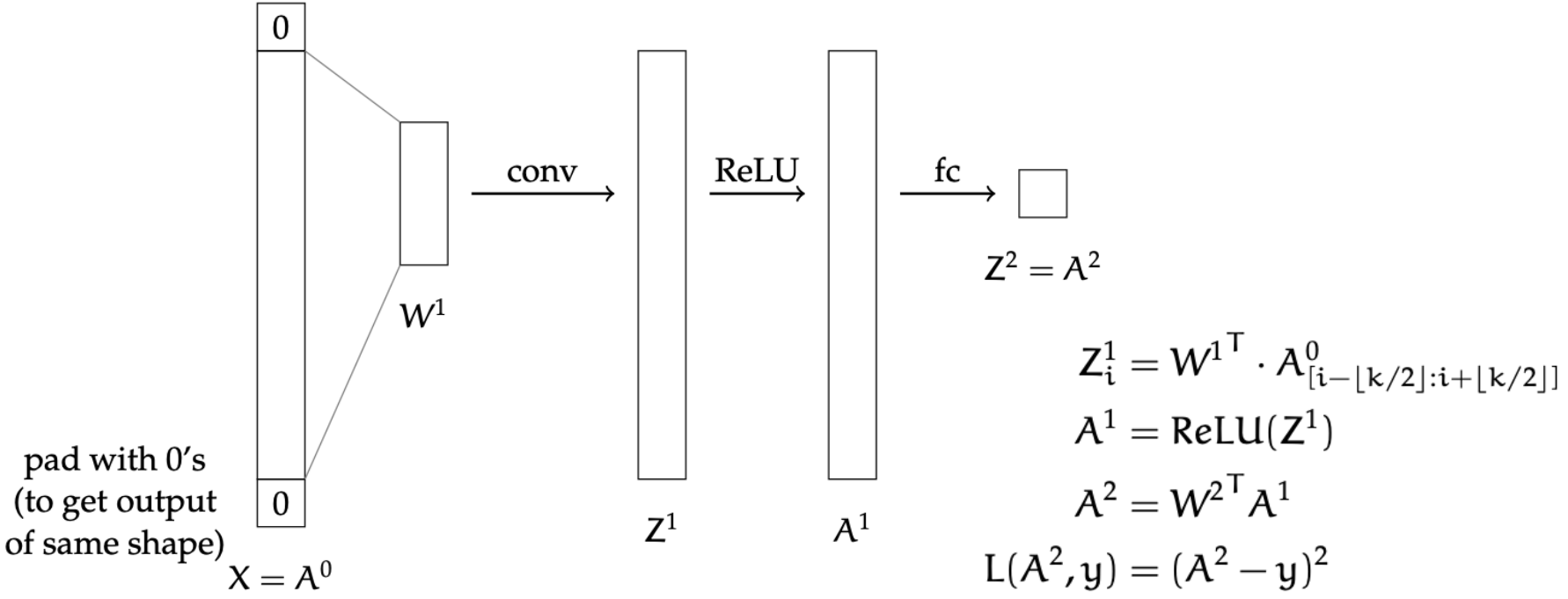
```python
import torch
import torch.nn as nn

def generate_model():
    # input: 1 × 28 × 28 (grayscale image)
    model = nn.Sequential(
        # first conv block
        nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2),

        # second conv block
        nn.Conv2d(32, 64, kernel_size=3),
        nn.ReLU(),
        nn.MaxPool2d(2, 2),

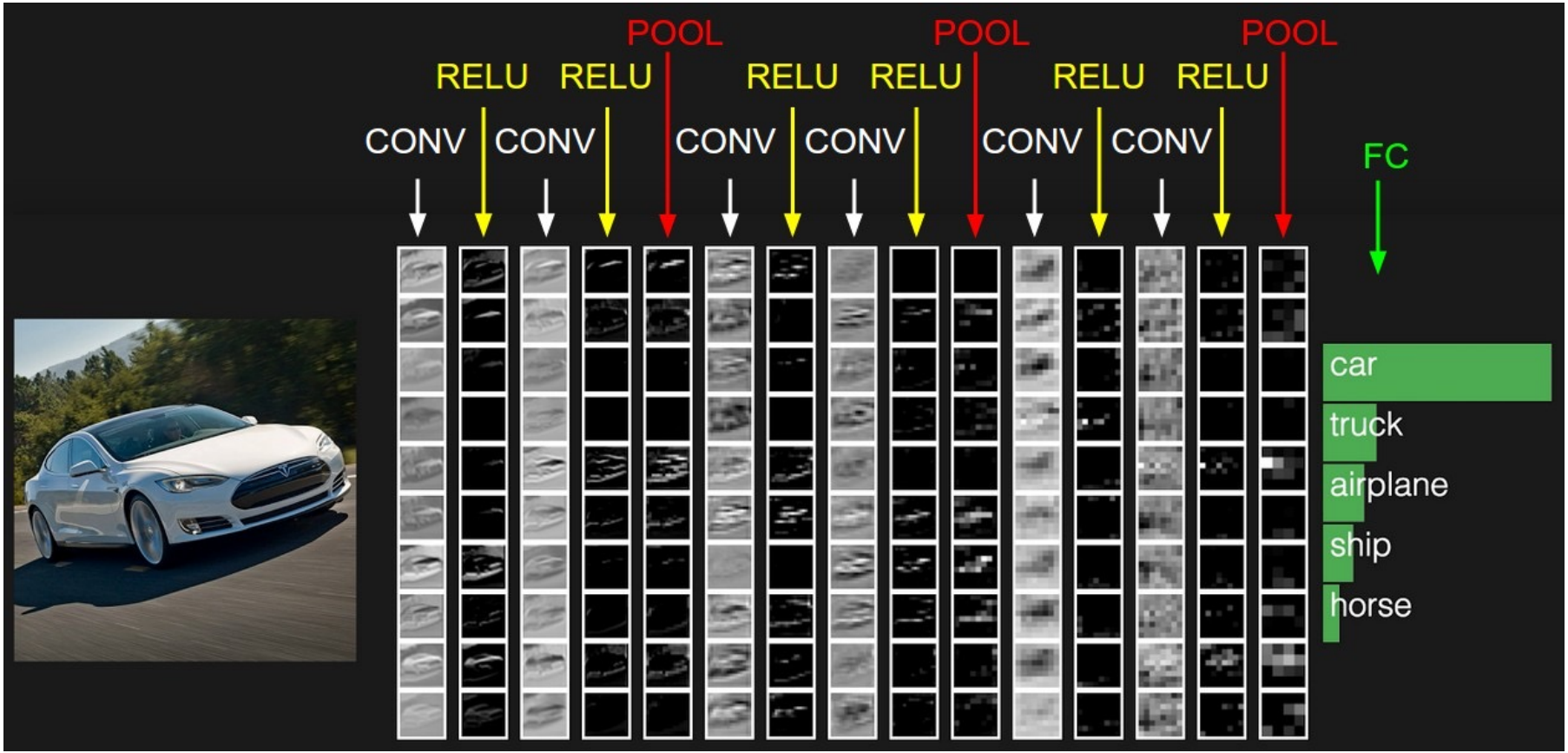
        # classifier
        nn.Flatten(),
        nn.Linear(64 * 5 * 5, 128),
        nn.ReLU(),
        nn.Linear(128, 10),
        nn.Softmax(dim=1)
    )
    return model
```





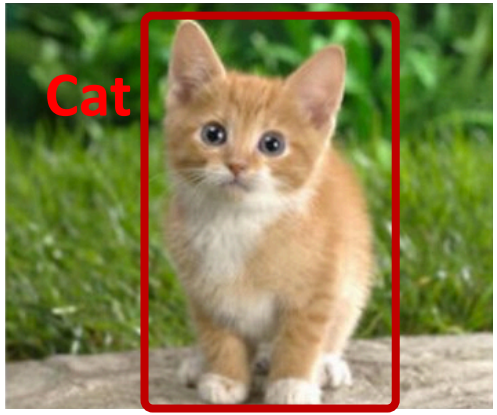
Backpropagation: $$\frac{\partial \text{loss}}{\partial W^1} = \frac{\partial Z^1}{\partial W^1} \cdot \frac{\partial A^1}{\partial Z^1} \cdot \frac{\partial \text{loss}}{\partial A^1}$$

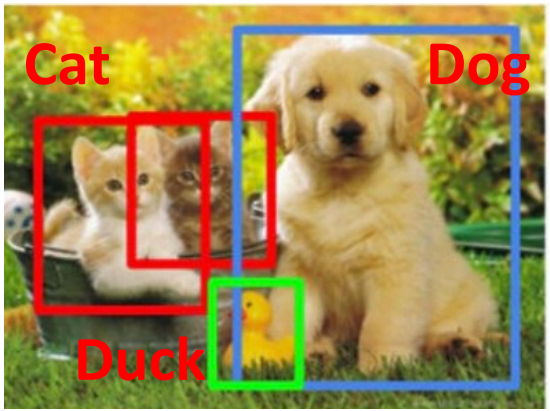
The activations of an example ConvNet architecture



Object detection







Object detection

Classification: C classes
Input: Image
Output: Class label
Evaluation metric: Accuracy



CAT

Localization:
Input: Image
Output: Box in the image (x, y, w, h)
Evaluation metric: Intersection over Union



(x, y, w, h)

Classification + Localization: Do both

Input: image



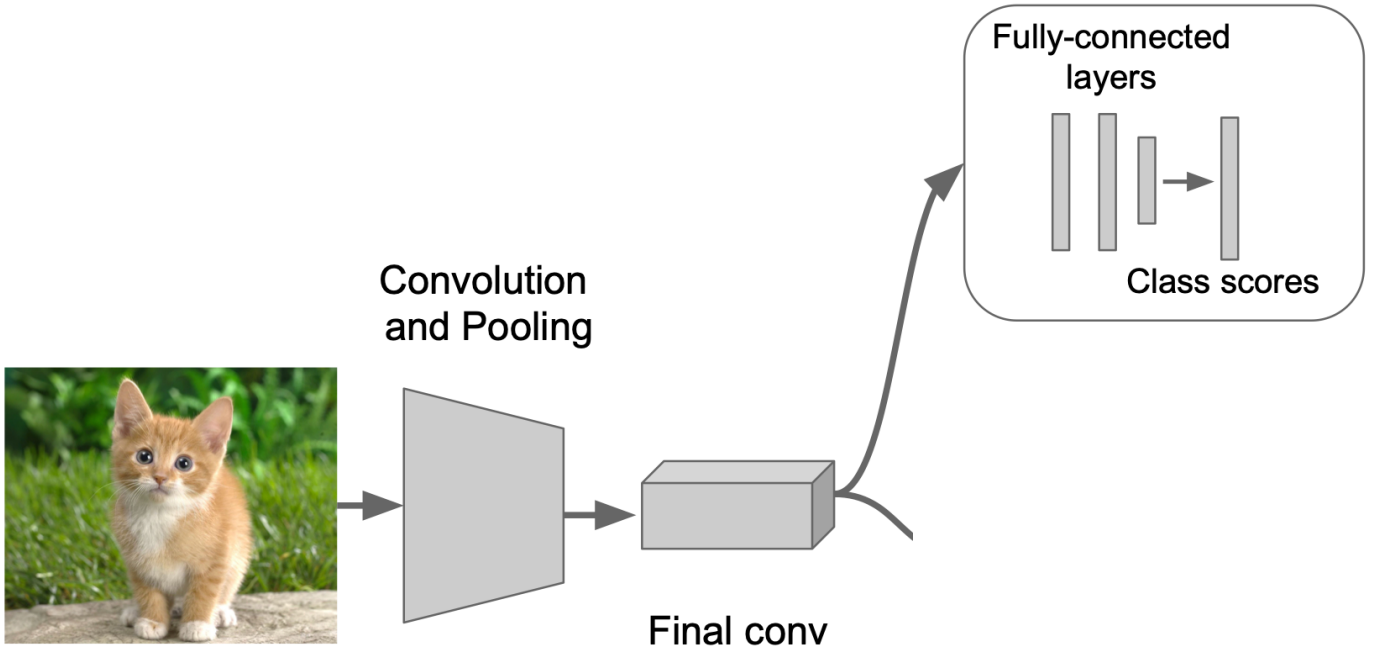
Only one object,
simpler than detection

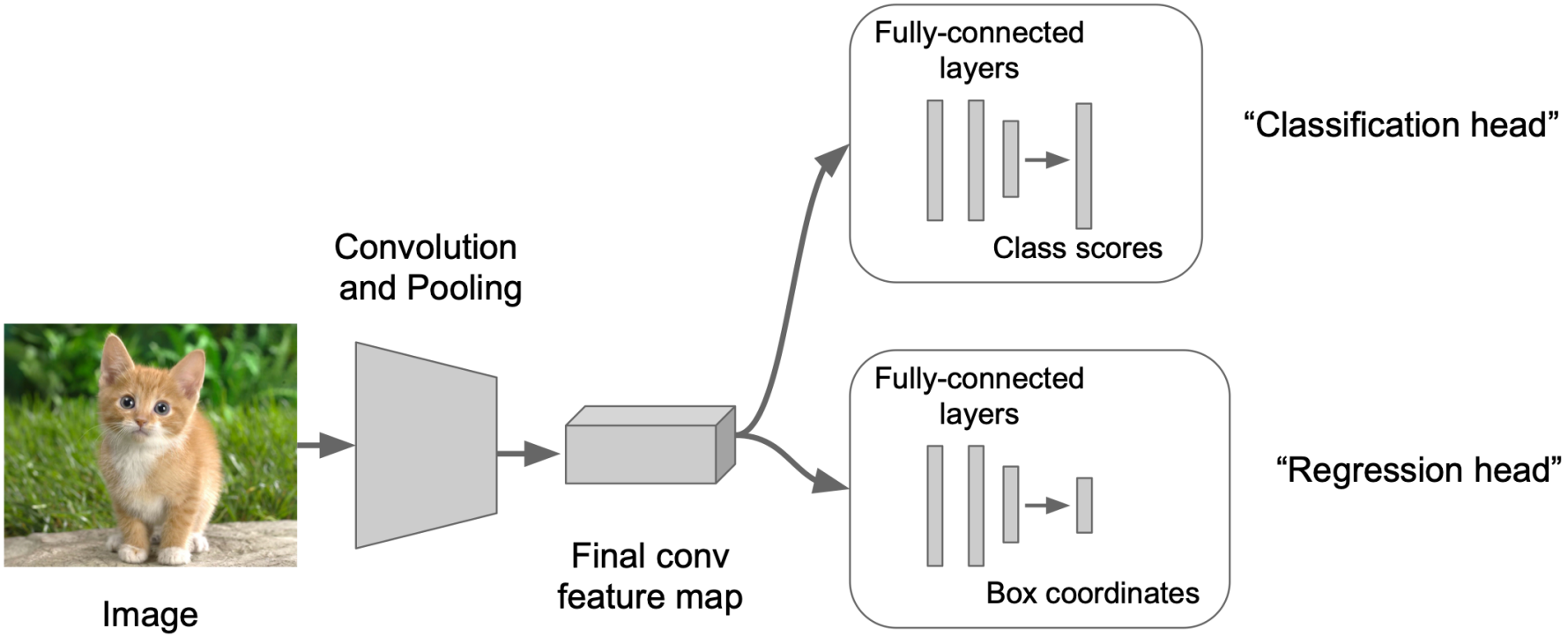
Neural Net

Output:
Box coordinates
(4 numbers)

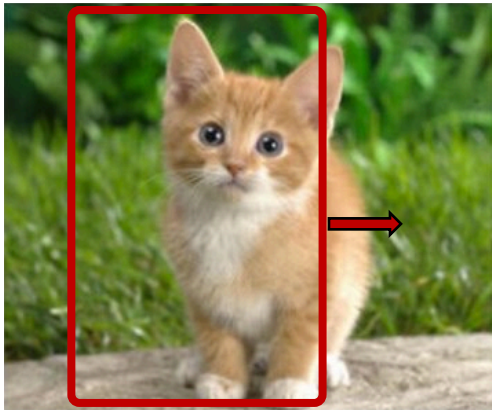
Correct output:
box coordinates
(4 numbers)

Loss:
L2 distance

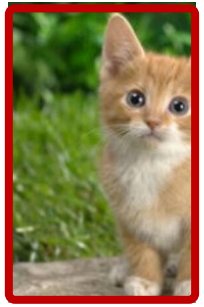

Fully-connected
layers
Class scores
“Classification head”
Convolution
and Pooling
Final conv
feature map
Image


Fully-connected layers
Class scores
"Classification head"
Convolution and Pooling
Fully-connected layers
Box coordinates
"Regression head"
Final conv feature map
Image

Sliding Window



Cat?



Cat?



A lot of regions to be checked



Image:
3 x 221 x 221

Convolution
+ pooling

Feature map:
1024 x 5 x 5

FC

4096

FC

4096

FC

Class scores:
1000

Softmax
loss

FC

4096

FC

1024

FC

Boxes:
1000 x 4

Euclidean
loss

Object detection



DOG, (x, y, w, h)
CAT, (x, y, w, h)
CAT, (x, y, w, h)
DUCK (x, y, w, h)

= 16 numbers





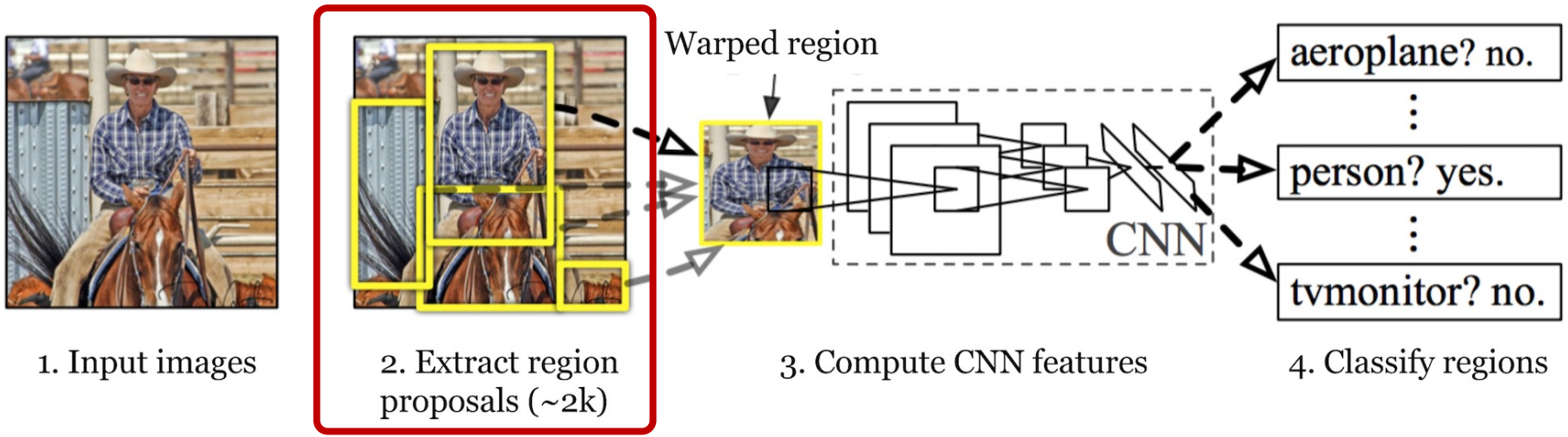
CNN → Class?

Way too many regions to be checked

Object detection

Girshick et al., 2014 (https://arxiv.org/abs/1311.2524)

R-CNN: Region-based Convolutional Neural Networks



Slow! Many regions; Cannot be used for real time inference

Manually defined region proposals

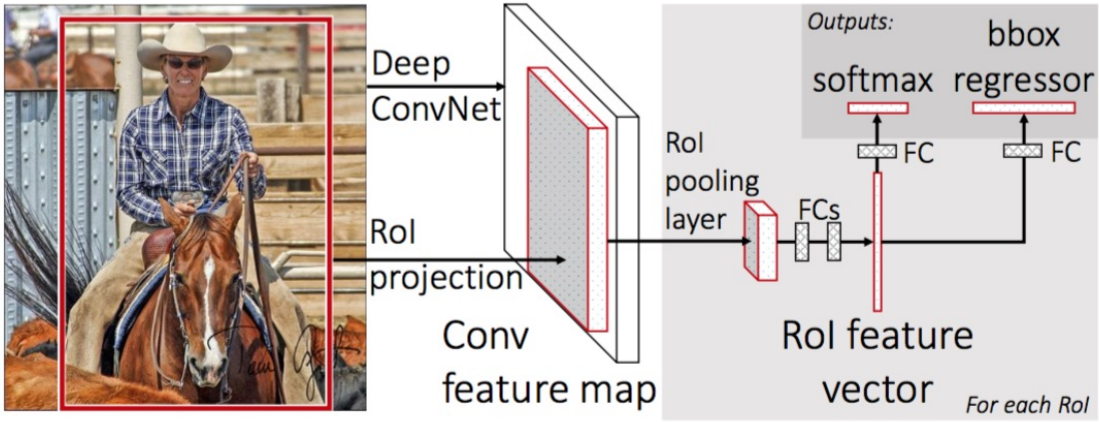
Uses the CNN network on each proposed region

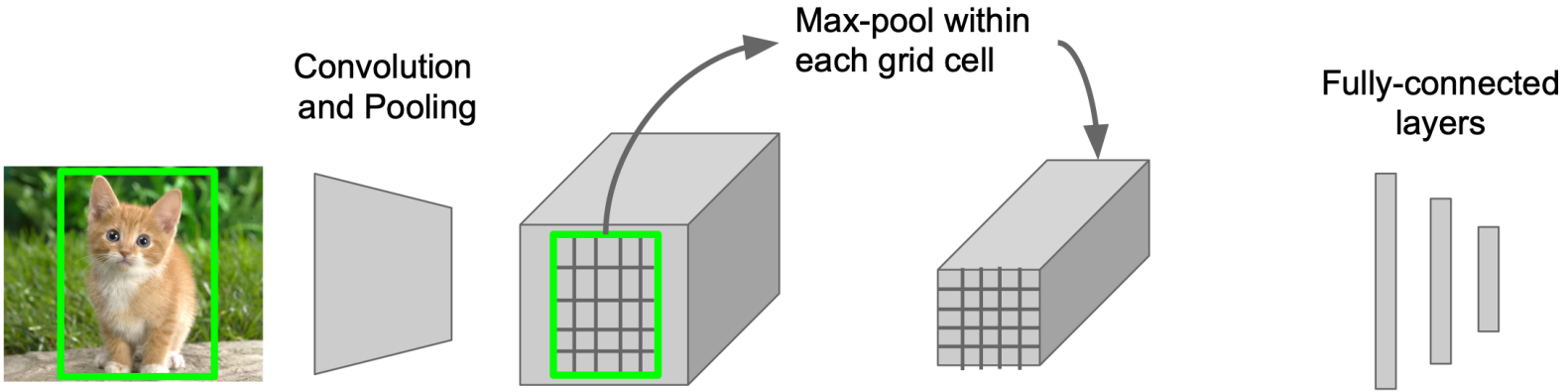
Object detection

Fast R-CNN Learns Region Proposals

https://arxiv.org/pdf/1504.08083

Uses the CNN network once on the image and classifies the region projections in the feature map.

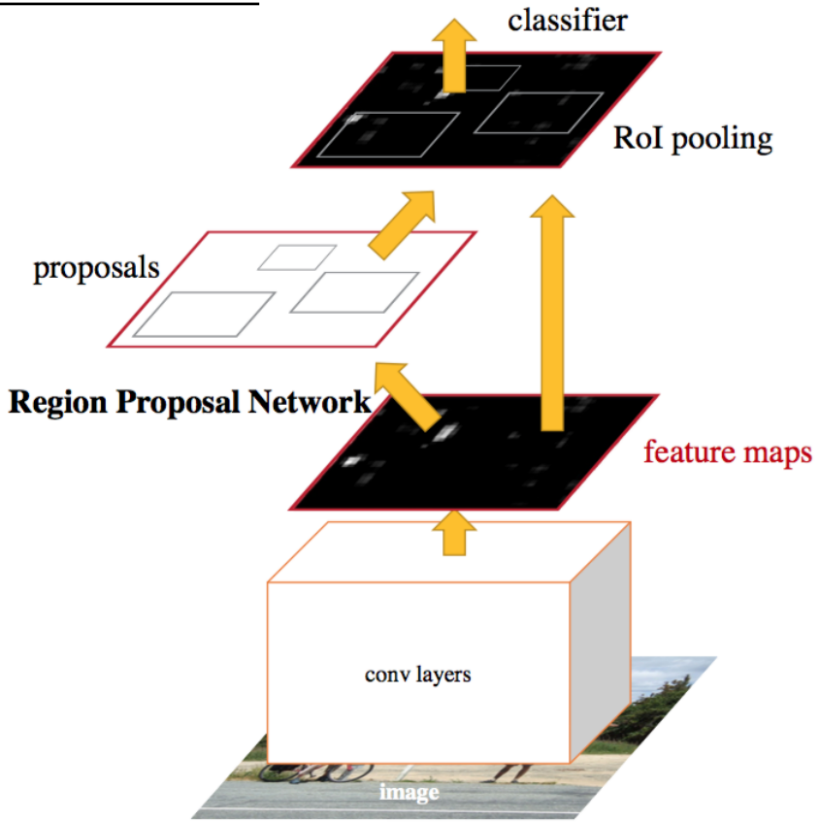




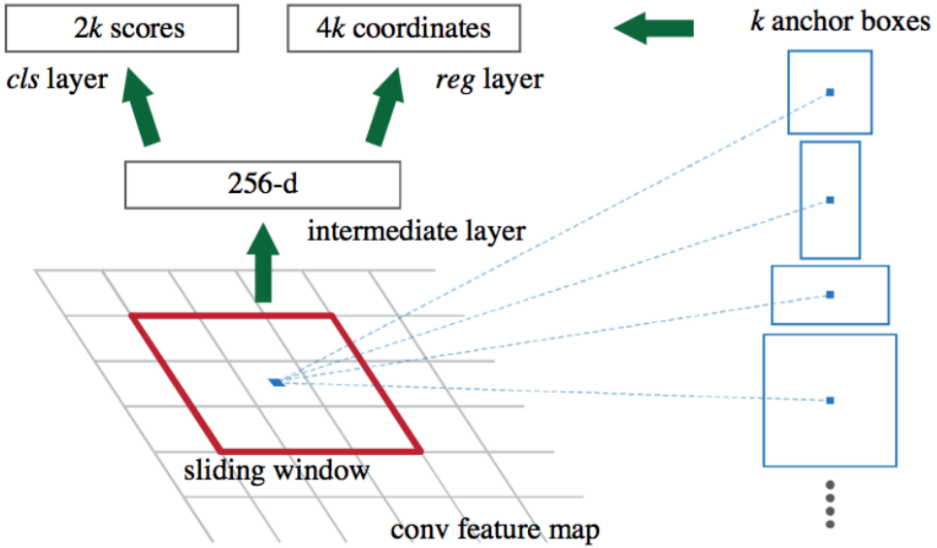
Object detection

https://arxiv.org/pdf/1506.01497

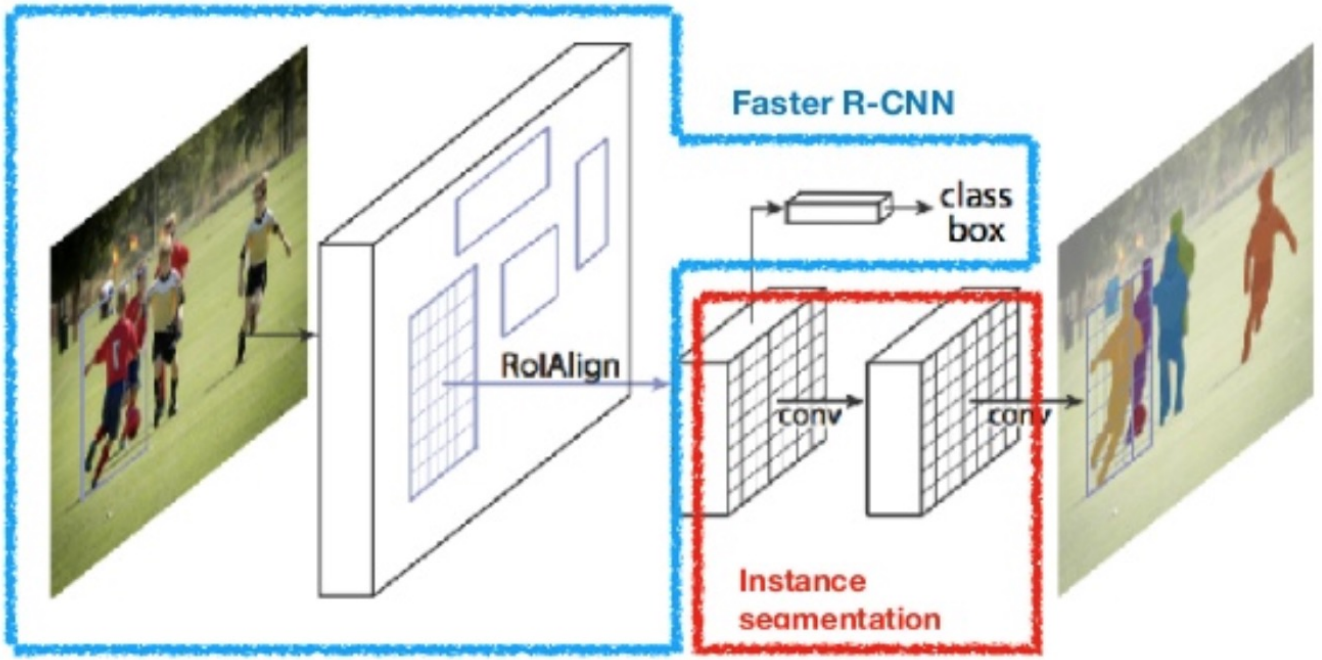
Faster R-CNN



Replaces the region proposal algorithm with a significantly faster neural network that can learn to propose better regions for the task at hand

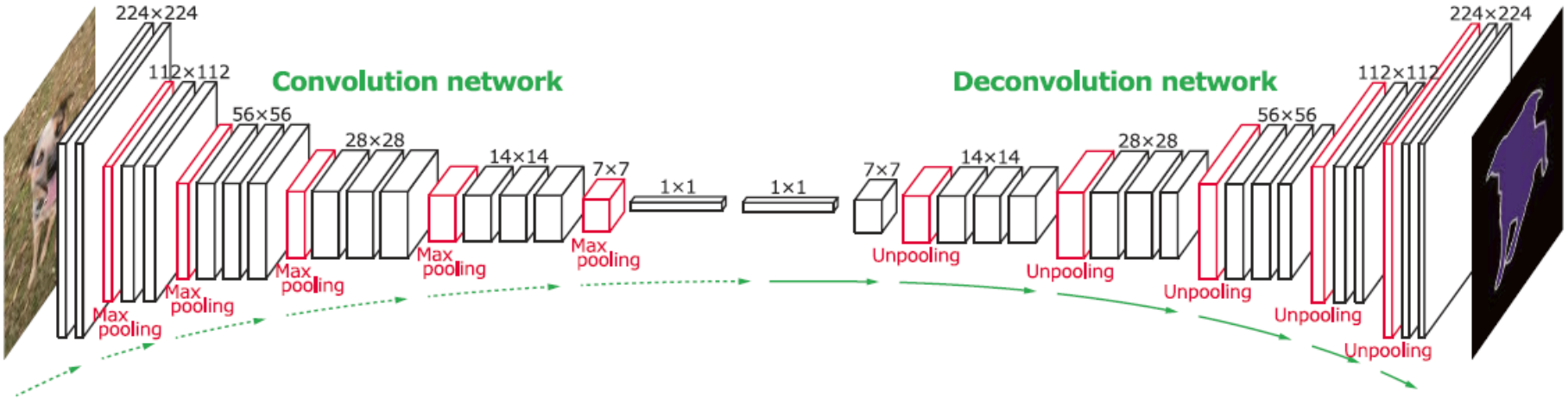


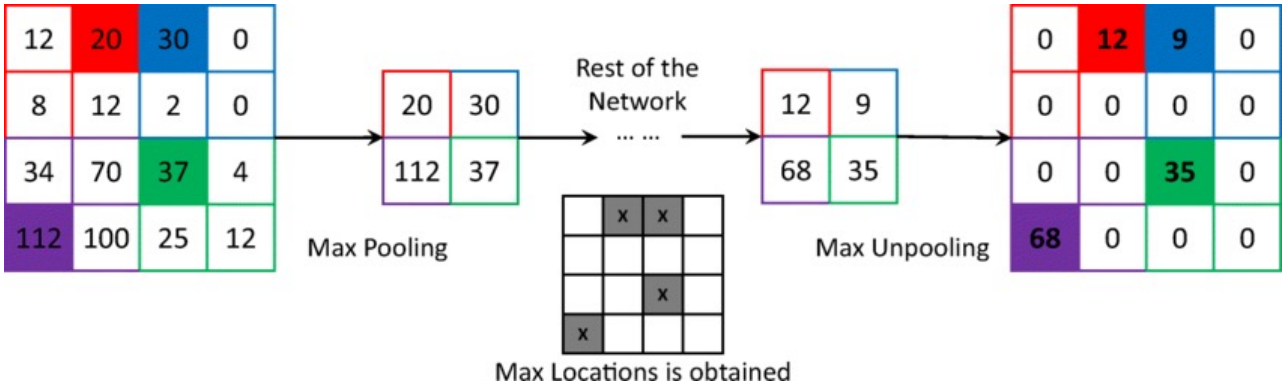
Segmentation



Mask R-CNN extends Faster R-CNN to pixel-level image segmentation. The key point is to decouple the classification and the pixel-level mask prediction task

Segmentation

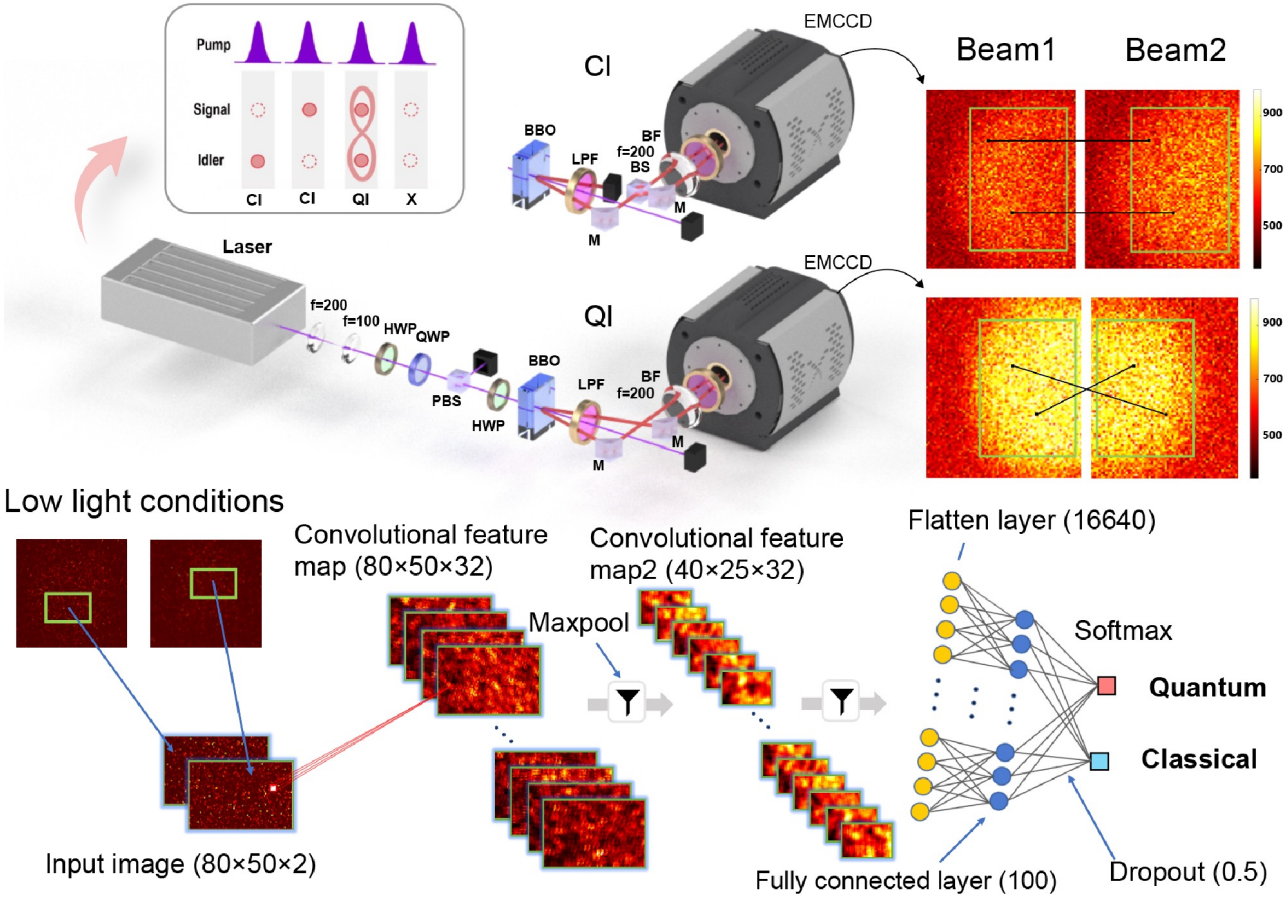




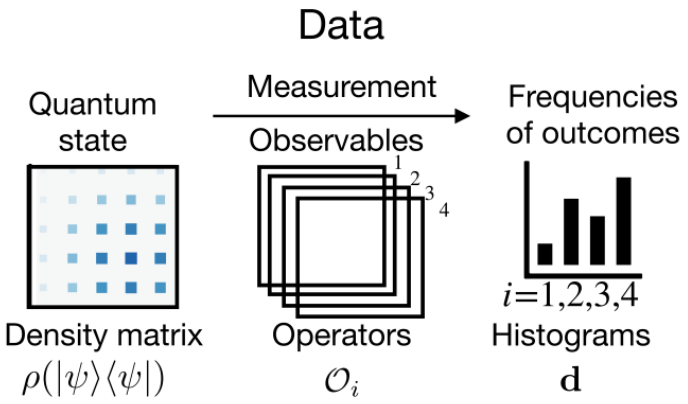
Some Examples in (Quantum) Physics

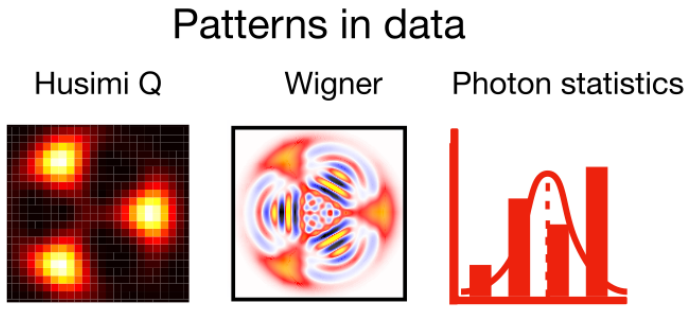
Classification of quantum correlation

https://doi.org/10.1364/OE.477046



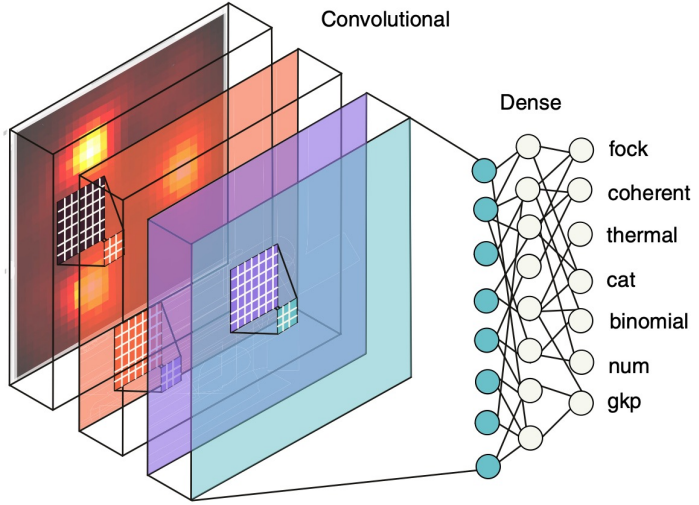
Classification of Quantum states or their properties





https://github.com/qutip/qutip

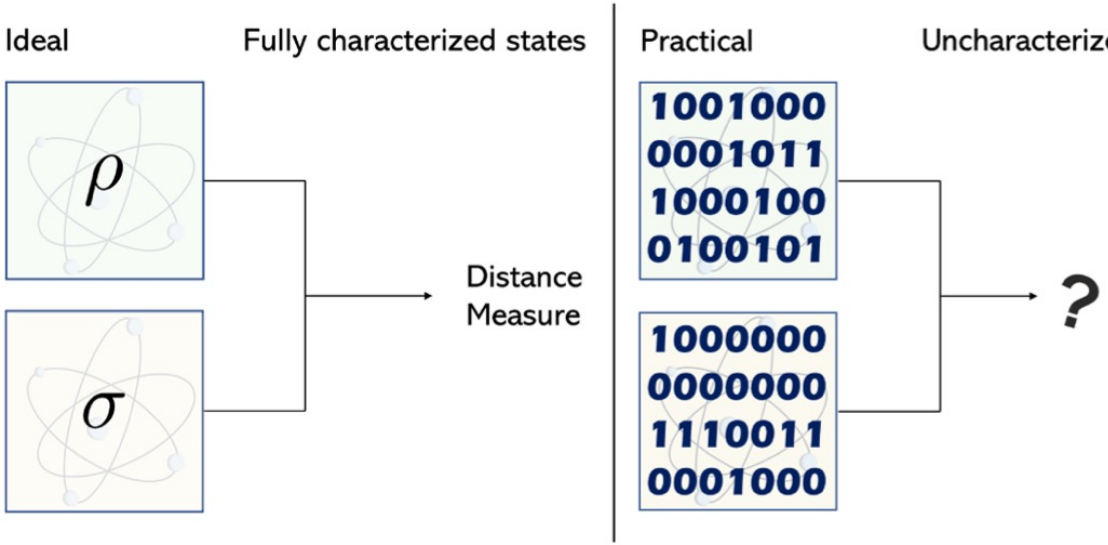
QuTip: Quantum Toolbox in Python

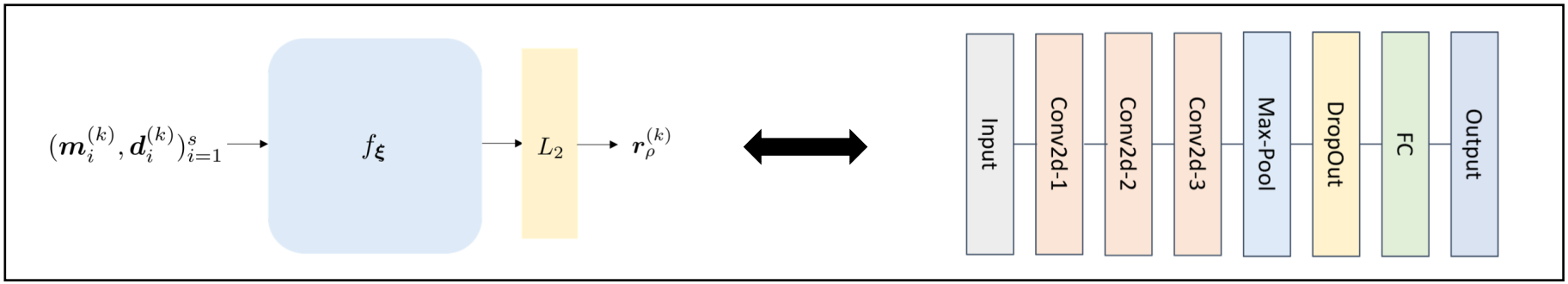


Layer	Output shape	No. of Parameters
Conv2D (32, 3, 1)	30, 30, 32	288
Conv2D (32, 3, 1)	28, 28, 32	9216
Conv2D (32, 5, 2)	14, 14, 32	9216
Conv2D (64, 3, 1)	12, 12, 64	18 432
Conv2D (64, 3, 1)	10, 10, 64	36 864
Conv2D (64, 5, 2)	5, 5, 64	36 864
Dense	512	524 800
Dense	256	131 328
Dense, output y_i	7	1799
Total parameters		768 807

Assessing the similarity of characterized vs uncharacterized quantum states

https://doi.org/10.1103/PhysRevLett.130.210601





Assessing the similarity of characterized vs uncharacterized quantum states

https://doi.org/10.1103/PhysRevLett.130.210601

Algorithm 1: Training of StateNet for learning quantum state similarity.

Data: number of states in training set N, state measurement results $\{\{\chi_{\rho_i}^{(k)}\}_{k=1}^{K}\}_{i=1}^{N}$, maximum number of epochs E, learning rate δ

Initialize parameters $\boldsymbol{\xi}$ and randomly, $e = 0$;

while $e < E$ **do**

  Calculate the state representation $r_{\rho_i}^{(k)}$ for each state measurement result $\chi_{\rho_i}^{(k)}$;

  $\mathcal{L} = 0$;

  for $i_1 = 1$ **to** N **do**

    Randomly select K state representations $r_{\rho_j}^{(k)}$ from all state representations, where $j \neq i_1$, and denote them as $\{r_{\rho_{neg}}^{(k)}\}_{k=1}^{K}$;

    for $i_2 = 1$ **to** K **do**

      Select $r_{\rho_{i_1}}^{(i_2)}$ as a reference state representation;

      Calculate the Euclidean distance $\text{dist}_k := ||r_{\rho_{i_1}}^{(i_2)} - r_{\rho_{i_1}}^{(k)}||_2$ between $r_{\rho_{i_1}}^{(i_2)}$ and each $r_{\rho_{i_1}}^{(k)}$, where $k \in \{1, 2, \cdots, K\}$;

      Calculate the Euclidean distance $||r_{\rho_{i_1}}^{(i_2)} - r_{\rho_{neg}}^{(k)}||_2$ between $r_{\rho_{i_1}}^{(i_2)}$ and each element in $\{r_{\rho_{neg}}^{(k)}\}_{k=1}^{K}$ and find the minimum $\text{dist}_{\text{neg}} := \min_k ||r_{\rho_{i_1}}^{(i_2)} - r_{\rho_{neg}}^{(k)}||_2$;

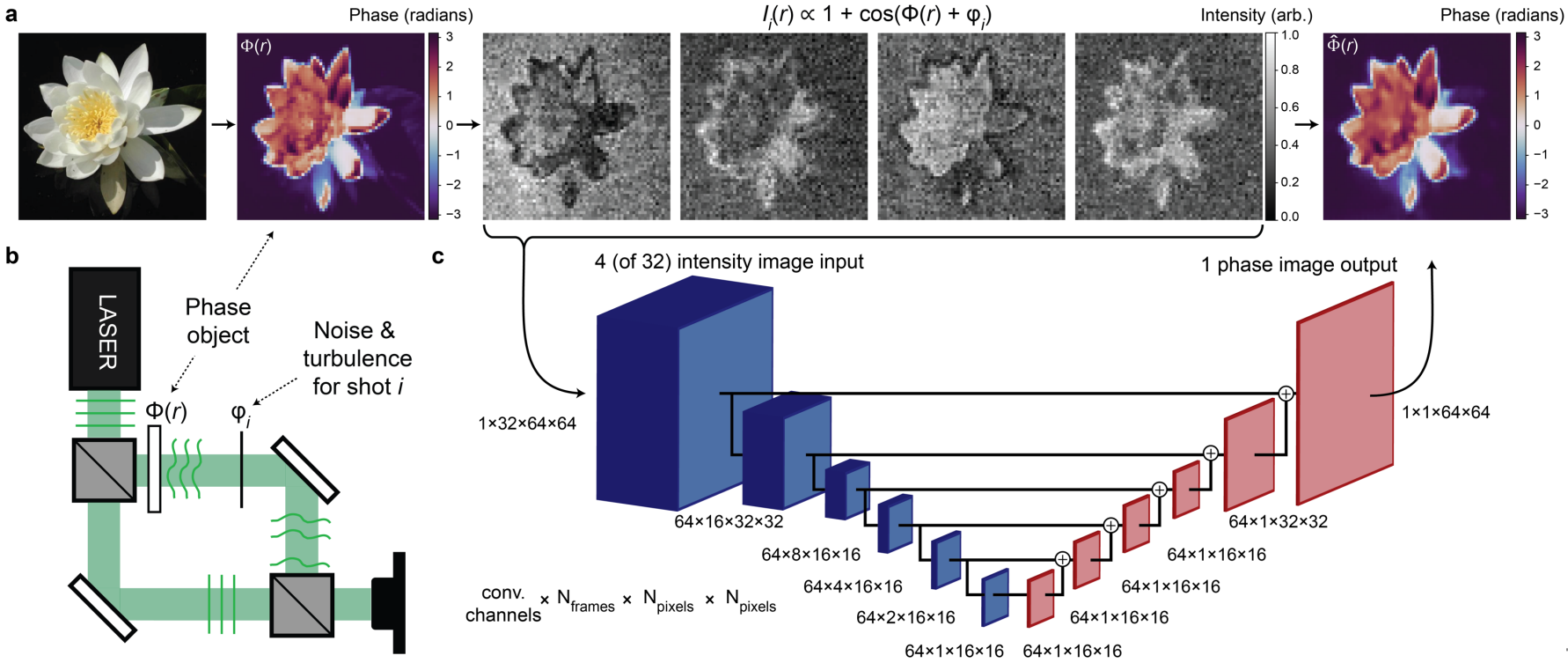
      $\mathcal{L} = \mathcal{L} + \sum_{k=1}^{K} \text{dist}_k - K \cdot \text{dist}_{\text{neg}}$;

  Update $\boldsymbol{\xi}$ as $\boldsymbol{\xi} = \boldsymbol{\xi} - \delta \nabla_{\boldsymbol{\xi}} \mathcal{L}$;

  $e = e + 1$;

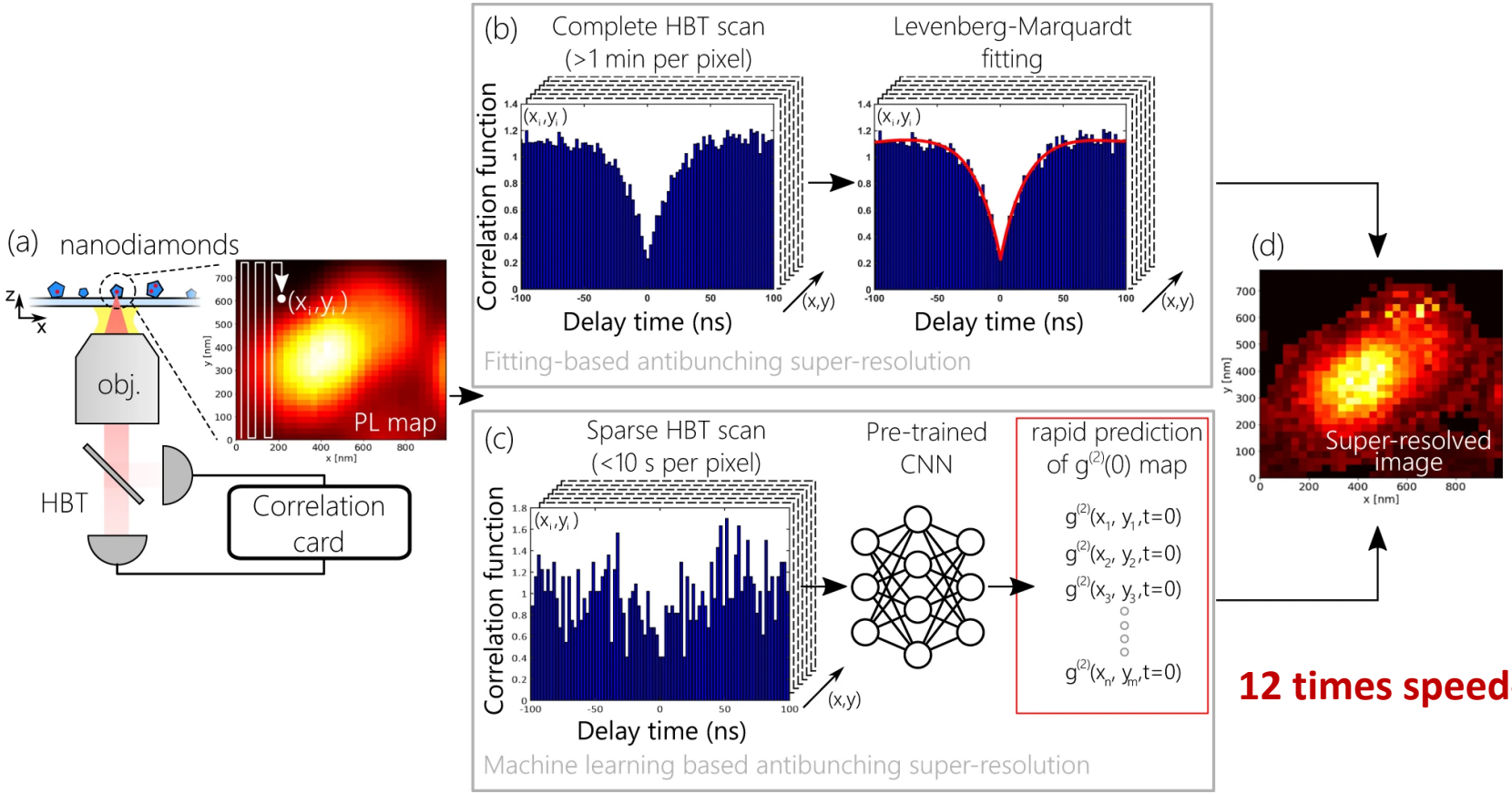
https://www.arxiv.org/abs/2506.13488
https://doi.org/10.1063/5.0204212

phase-image reconstructions from inputs consisting of noisy, randomly phase-modulated frames

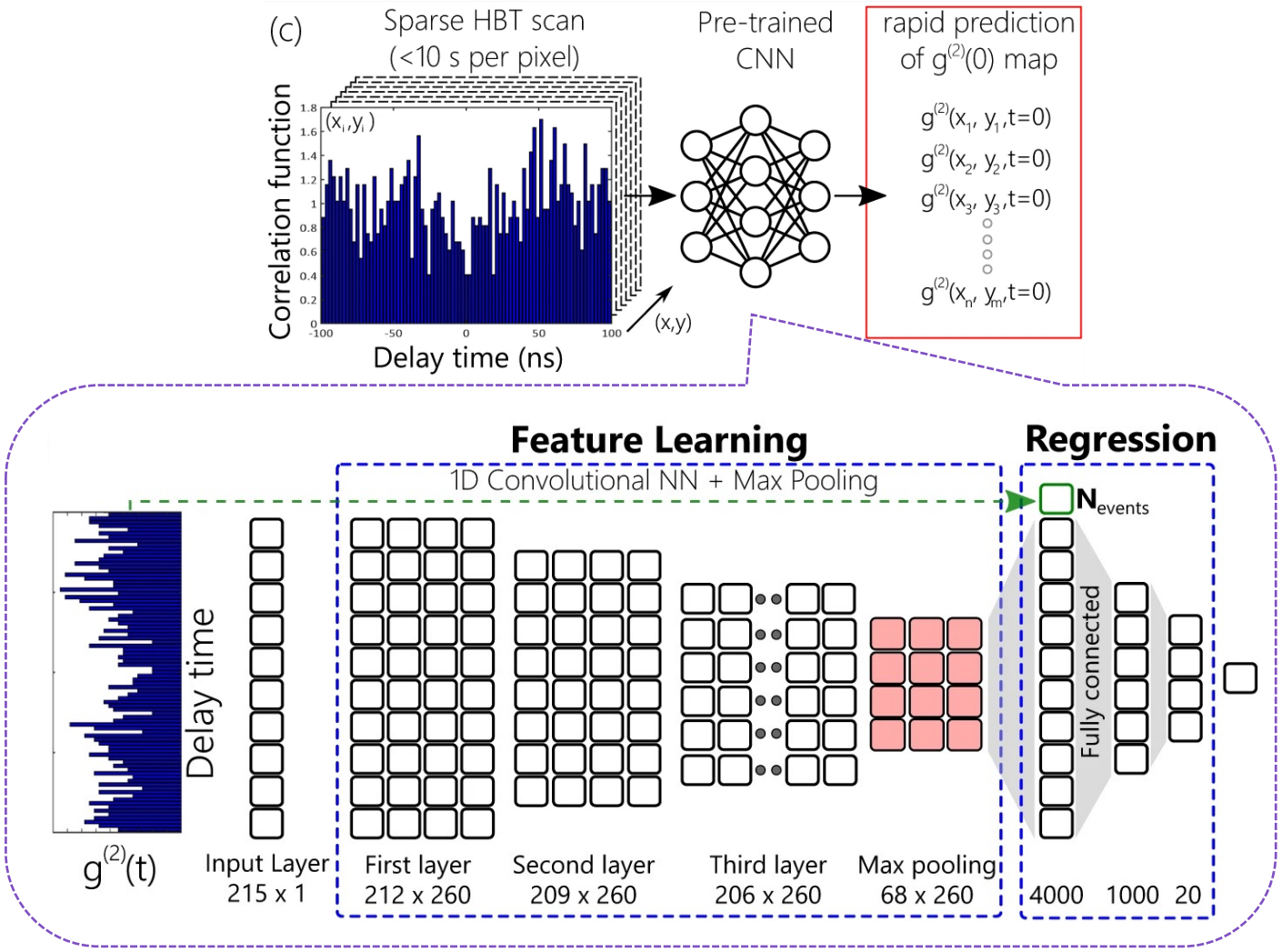


rapid anti-bunching super-resolution imaging

https://www.nature.com/articles/s41467-023-40506-4

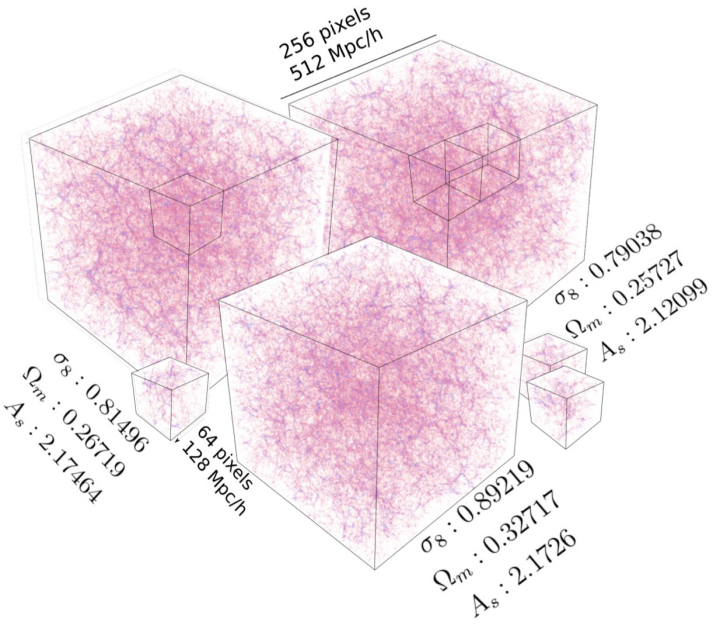


12 times speed-up

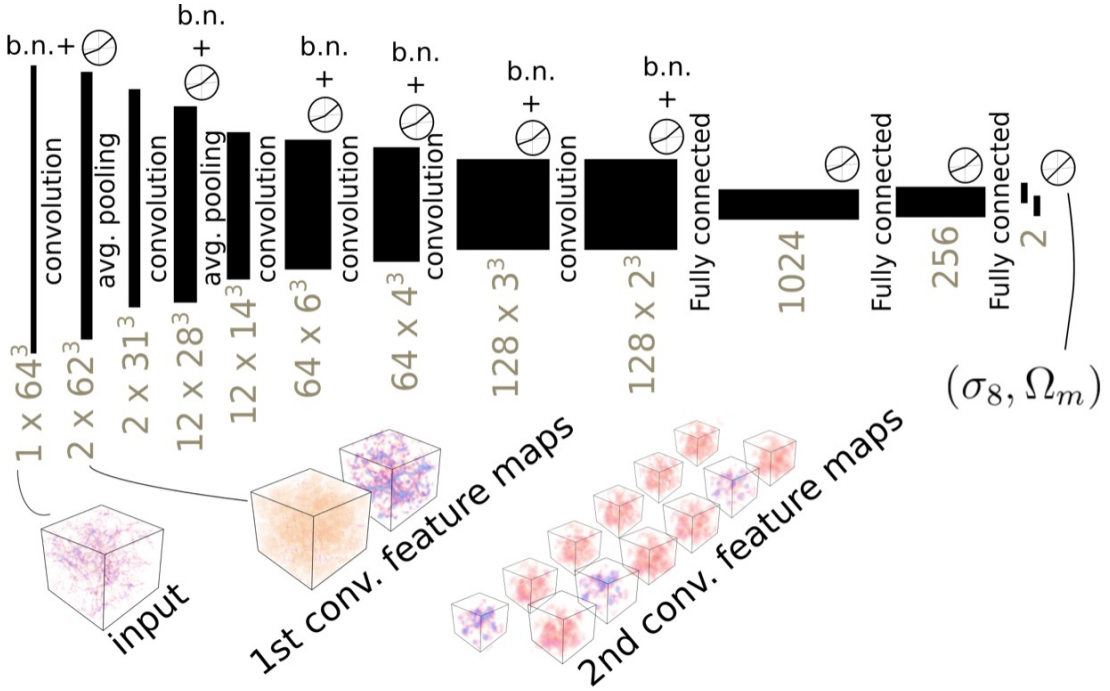

(c)
Sparse HBT scan
(<10 s per pixel)
Correlation function
Delay time (ns)
(x,y)
Pre-trained
CNN
rapid prediction
of g(2)(0) map
g(2)(x1, y1,t=0)
g(2)(x2, y2,t=0)
g(2)(x3, y3,t=0)
g(2)(xn, ym,t=0)
Feature Learning
Regression
1D Convolutional NN + Max Pooling
Nevents
Fully connected
Delay time
g(2)(t)
Input Layer
215 x 1
First layer
212 x 260
Second layer
209 x 260
Third layer
206 x 260
Max pooling
68 x 260
4000
1000
20

estimate cosmological parameters from the distribution of matter

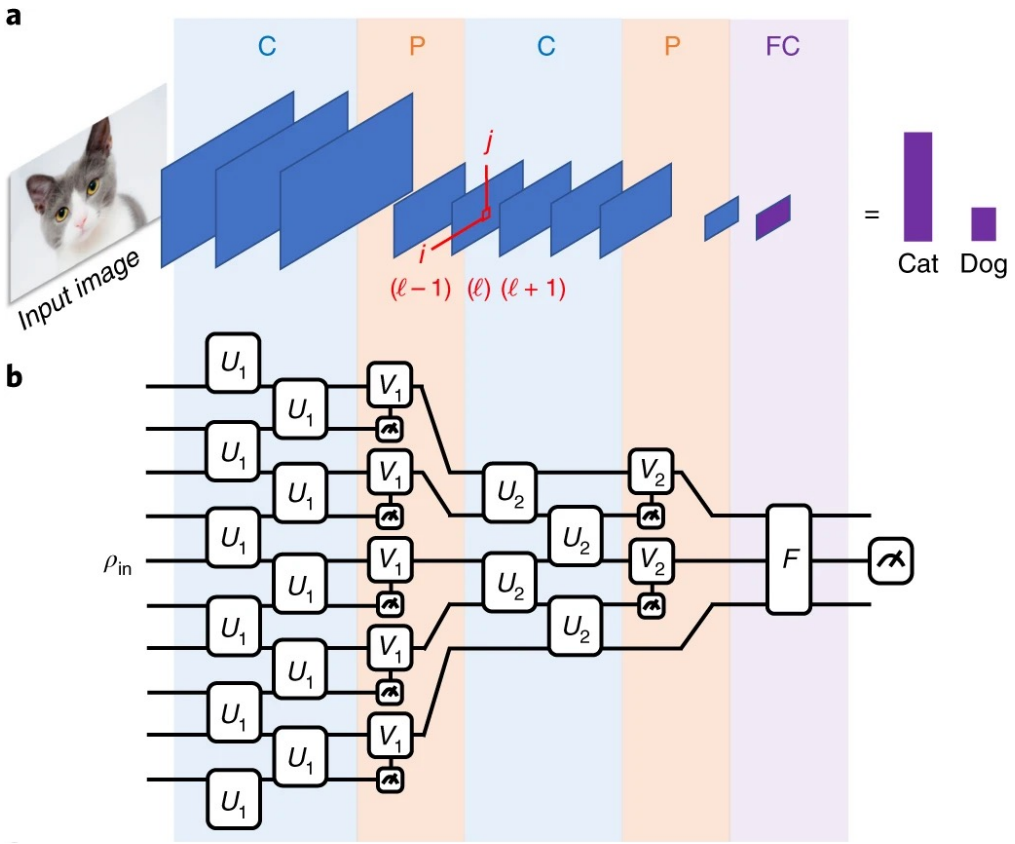
https://arxiv.org/abs/1711.02033



Dark matter distribution in three cubes produced using different sets of parameters. Each cube is divided into small sub-cubes for training and prediction

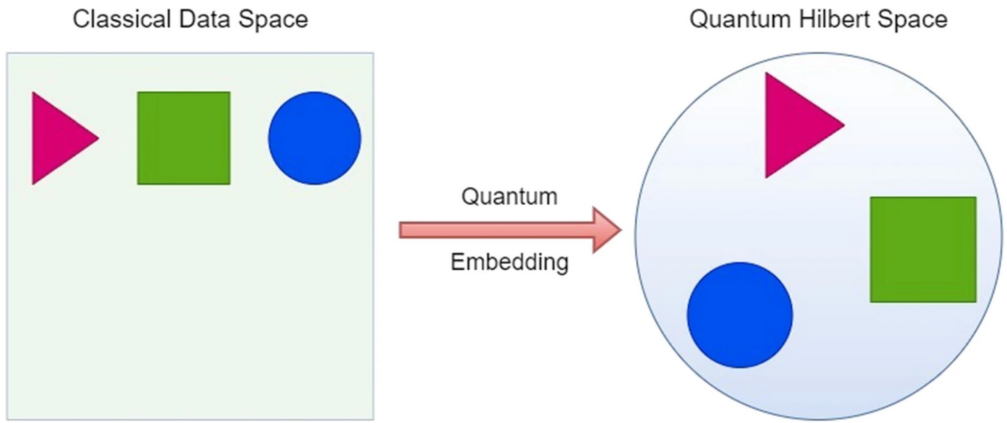


Quantum Convolutional Neural Networks



Nat. Phys. 15, 1273–1278 (2019)

Quantum Convolutional Neural Networks

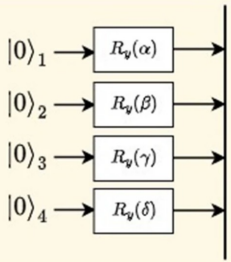


Two-qubit Amplitude Encoding:

$$|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$$

Classical data features

Four-qubit Angle Encoding:





Do CNNs actually learn physics, or just correlations in images?

Can we claim “interpretability” or is it pattern-matching with no physical understanding?

Do Pooling Layers Destroy Physical Information?

Do you think CNNs are only useful for pictures? Why or why not?